

+ 26

Back To Basics

Lambdas, Function Objects, and `std::function`

ROTH MICHAELS



20
26



Who is talking

Roth Michaels

Director of Software Research
inMusic



inMusic

Inspiring Creativity Worldwide

air

AKAI
PROFESSIONAL

mog

NI NATIVE
INSTRUMENTS

Numark

RANE

ALESIS

ALESIS
— DRUMS —

DENON DJ

BFD

engine dj

M-AUDIO

ALTO
PROFESSIONAL

ION

HEADRUSH

Sound
Switch













You can do it!



How I got here



Bell Labs

```
<CsoundSynthesizer>

<CsOptions>
  csound -W -d -o tone.wav
</CsOptions>

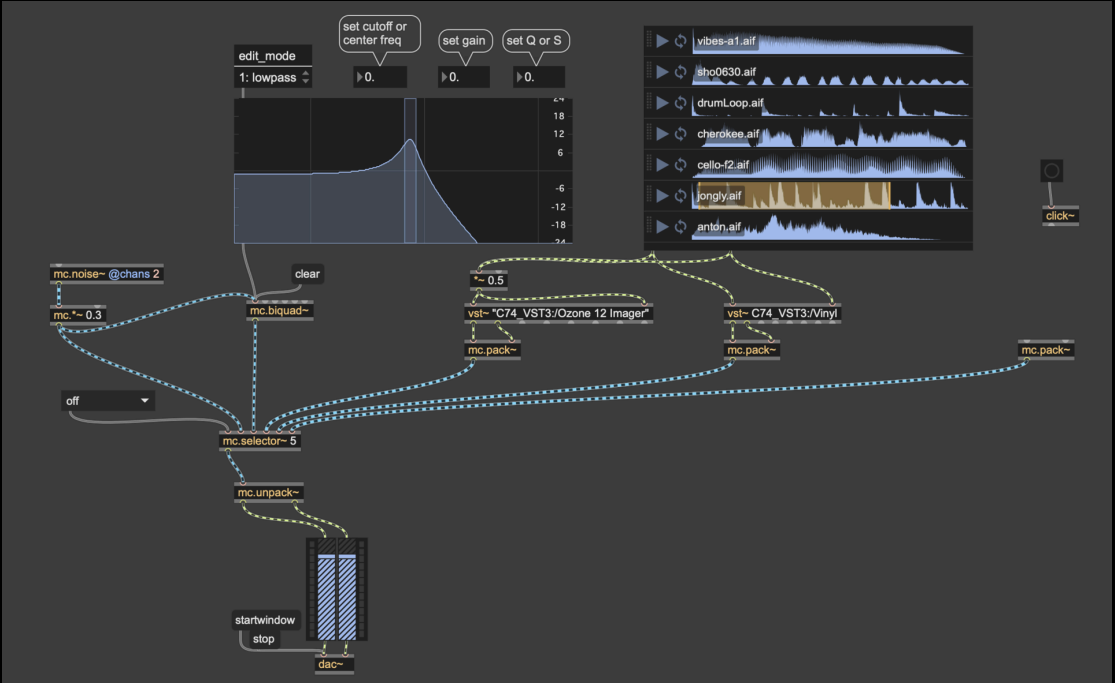
<CsInstruments>
  sr      = 96000      ; Sample rate.
  kr      = 9600       ; Control signal rate.
  ksmps   = 10         ; Samples per control signal.
  nchnls  = 1          ; Number of output channels.

  instr 1
    a1    oscil p4, p5, 1 ; Oscillator: p4 and p5 are the arguments from the score, 1 is
the table number.
    out a1 ; Output.
  endin
</CsInstruments>

<CsScore>
  f1 0 8192 10 1 ; Table containing a sine wave. Built-in generator 10 produces a
sum of sinusoids, here only one.
  i1 0 1 20000 1000 ; Play one second of one kHz at amplitude 20000.
  e
</CsScore>

</CsoundSynthesizer>
```

Csound



Max/MSP

Bell Laboratories at New Providence, New Jersey. By Ken Lund from Reno, Nevada, USA, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=146559106>

+ 26

Back To Basics

Lambdas, Function Objects, and `std::function`

ROTH MICHAELS



20
26



What we make

- Hardware and software for digital audio
 - mixing and mastering
 - producing beats
 - DJing
 - post for film, television and radio
- The examples in this talk come from that work
 - you do not need to know any audio or DSP


```
#include <c++>
```

- The C++ community's diversity and inclusion group
 - <https://www.incldecpp.org>
- Come and say hello

Let's make this talk interactive

...please ask questions

Where we are going

- What is a function, really?
 - and what does the Core Guidelines say about passing one around
- Function pointers, member functions, pointers to members
- **Function objects:** the thing with `operator()`
- **Lambdas:** function objects with a friendlier face
- `std::function`: type erasure, and what it costs
- Bugs I have actually shipped
- Bonus: Objective-C blocks

What is a function?

We are going to need to agree on this

In mathematics

```
// In maths, a function is a mapping from a domain to a codomain:  
//  
//       $f(x) = 2x + 1$   
//  
// Same input, same output, no other effect.  
constexpr int f(int x) {  
    return 2 * x + 1;  
}  
  
static_assert(f(20) == 41);
```


In mathematics

```
// In maths, a function is a mapping from a domain to a codomain:  
//  
//       $f(x) = 2x + 1$   
//  
// Same input, same output, no other effect.  
constexpr int f(int x) {  
    return 2 * x + 1;  
}  
  
static_assert(f(20) == 41);
```


In mathematics

```
// In maths, a function is a mapping from a domain to a codomain:  
//  
//       $f(x) = 2x + 1$   
//  
// Same input, same output, no other effect.  
constexpr int f(int x) {  
    return 2 * x + 1;  
}  
  
static_assert(f(20) == 41);
```


In mathematics

```
// In maths, a function is a mapping from a domain to a codomain:  
//  
//       $f(x) = 2x + 1$   
//  
// Same input, same output, no other effect.  
constexpr int f(int x) {  
    return 2 * x + 1;  
}  
  
static_assert(f(20) == 41);
```


In mathematics

```
// In maths, a function is a mapping from a domain to a codomain:  
//  
//       $f(x) = 2x + 1$   
//  
// Same input, same output, no other effect.  
constexpr int f(int x) {  
    return 2 * x + 1;  
}
```

```
static_assert(f(20) == 41);
```


In code, we use the same word for this

```
// In code we use the same word for something that is not a mapping at
// all. It returns nothing; it exists only for its effect.
void log_line(const std::string& message) {
    std::printf("%s\n", message.c_str());
}
```


In code, we use the same word for this

```
// In code we use the same word for something that is not a mapping at
// all. It returns nothing; it exists only for its effect.
void log_line(const std::string& message) {
    std::printf("%s\n", message.c_str());
}
```


In code, we use the same word for this

```
// In code we use the same word for something that is not a mapping at  
// all. It returns nothing; it exists only for its effect.  
void log_line(const std::string& message) {  
    std::printf("%s\n", message.c_str());  
}
```


A function with no parameters

```
// A function with no parameters cannot be a mapping either...
```

```
int roll_a_die();
```

```
// ...unless it is a constant.
```

```
constexpr int answer() {
```

```
    return 42;
```

```
}
```

```
static_assert(answer() == 42);
```


A function with no parameters

```
// A function with no parameters cannot be a mapping either...
```

```
int roll_a_die();
```

```
// ...unless it is a constant.
```

```
constexpr int answer() {
```

```
    return 42;
```

```
}
```

```
static_assert(answer() == 42);
```


A function with no parameters

```
// A function with no parameters cannot be a mapping either...  
int roll_a_die();  
  
// ...unless it is a constant.  
constexpr int answer() {  
    return 42;  
}  
  
static_assert(answer() == 42);
```


A function with no parameters

```
// A function with no parameters cannot be a mapping either...
int roll_a_die();

// ...unless it is a constant.
constexpr int answer() {
    return 42;
}

static_assert(answer() == 42);
```

In C++

```
// return type      name      parameters
//      |            |            |
//      v            v            v

double decibels_to_gain(double decibels) {
    return std::pow(10.0, decibels / 20.0);
}
```


...and at least six things get called "a function"

- A free function
- A member function — with an invisible extra parameter
- A **static** member function — which is just a free function in a hat
- A **function pointer** — an object that refers to one
- A **function object** — a class with `operator()`
- A **lambda closure** — which is a function object the compiler wrote
- A **type-erased wrapper** — `std::function` and its relatives

callable

How do I pass one?
...and while we are here, how do I pass anything?

F.15: Prefer simple and conventional ways of passing information

- "Using *unusual and clever* techniques causes surprises, slows understanding by other programmers, and encourages bugs."
- "If you really feel the need for an optimization beyond the common techniques, **measure** to ensure that it really is an improvement, and **document/comment**, because the improvement might not be portable."

F.15: normal parameter passing

	Cheap or impossible to copy (e.g., int, unique_ptr)	Cheap to move (e.g., vector<T>, string) or Moderate cost to move (e.g., array<vector>, BigPOD) or Don't know (e.g., unfamiliar type, template)	Expensive to move (e.g., BigPOD[], array<BigPOD>)
Out	X f()		
In/Out	f(X&)		
In	f(X)	f(const X&)	
In & retain "copy"			

"Cheap" ≈ a handful of hot int copies
"Moderate cost" ≈ memcpy hot/contiguous ~1KB and no allocation

** or return unique_ptr<X>/make_shared_<X> at the cost of a dynamic allocation*

`isocpp.github.io/CppCoreGuidelines#Rf-conventional`

F.15: advanced parameter passing

	Cheap or impossible to copy (e.g., int, unique_ptr)	Cheap to move (e.g., vector<T>, string) or Moderate cost to move (e.g., array<vector>, BigPOD) or Don't know (e.g., unfamiliar type, template)	Expensive to move (e.g., BigPOD[], array<BigPOD>)
Out	X f()		
In/Out	f(X&)		
In	f(X)	f(const X&)	
In & retain copy		f(const X&) + f(X&&) & move	**
In & move from		f(X&&)	**

** or return unique_ptr<X>/make_shared_<X> at the cost of a dynamic allocation*

*** special cases can also use perfect forwarding (e.g., multiple in+copy params, conversions)*

the green cells are what is new

The normal table, in code

```
// "in", cheap to copy  
void set_gain(float gain);  
  
// "in", expensive to copy  
void render(const std::vector<float>& buffer);  
  
// "in/out"  
void normalise(std::vector<float>& buffer);  
  
// "out"
```

The normal table, in code

```
// "in", cheap to copy  
void set_gain(float gain);
```

```
// "in", expensive to copy  
void render(const std::vector<float>& buffer);
```

```
// "in/out"  
void normalise(std::vector<float>& buffer);
```

```
// "out"
```


The normal table, in code

```
// "in", cheap to copy  
void set_gain(float gain);
```

```
// "in", expensive to copy  
void render(const std::vector<float>& buffer);
```

```
// "in/out"  
void normalise(std::vector<float>& buffer);
```

```
// "out"
```

The normal table, in code

```
// "in", cheap to copy  
void set_gain(float gain);
```

```
// "in", expensive to copy  
void render(const std::vector<float>& buffer);
```

```
// "in/out"  
void normalise(std::vector<float>& buffer);
```

```
// "out"
```


The normal table, in code

```
// "in", cheap to copy  
void set_gain(float gain);  
  
// "in", expensive to copy  
void render(const std::vector<float>& buffer);  
  
// "in/out"  
void normalise(std::vector<float>& buffer);  
  
// "out"
```

The advanced table, in code

```
// "in & retain a copy" -- sink the argument
class Track {
public:
    explicit Track(std::string name) : m_name{std::move(name)} {}

private:
    std::string m_name;
};

// "in & move from" -- the caller must have finished with it
void consume(std::vector<float>&& buffer);

// "forward" -- pass on exactly what we were handed
template <class T> void relay(T&& thing);
```


The advanced table, in code

```
// "in & retain a copy" -- sink the argument
class Track {
public:
    explicit Track(std::string name) : m_name{std::move(name)} {}

private:
    std::string m_name;
};

// "in & move from" -- the caller must have finished with it
void consume(std::vector<float>&& buffer);

// "forward" -- pass on exactly what we were handed
template <class T> void relay(T&& thing);
```

The advanced table, in code

```
// "in & retain a copy" -- sink the argument
class Track {
public:
    explicit Track(std::string name) : m_name{std::move(name)} {}

private:
    std::string m_name;
};

// "in & move from" -- the caller must have finished with it
void consume(std::vector<float>&& buffer);

// "forward" -- pass on exactly what we were handed
template <class T> void relay(T&& thing);
```


The advanced table, in code

```
// "in & retain a copy" -- sink the argument
class Track {
public:
    explicit Track(std::string name) : m_name{std::move(name)} {}

private:
    std::string m_name;
};

// "in & move from" -- the caller must have finished with it
void consume(std::vector<float>&& buffer);

// "forward" -- pass on exactly what we were handed
template <class T> void relay(T&& thing);
```

F.16 - F.21

- **F.16** For "in" parameters, pass cheaply-copied types by value and others by reference to `const`
- **F.17** For "in-out" parameters, pass by reference to non-`const`
- **F.18** For "will-move-from" parameters, pass by `X&&` and `std::move` the parameter
- **F.19** For "forward" parameters, pass by `TP&&` and only `std::forward` the parameter
- **F.20** For "out" output values, prefer return values to output parameters
- **F.21** To return multiple "out" values, prefer returning a struct

R.32 - R.36: and if it is a smart pointer

- **R.32** Take a `unique_ptr<widget>` parameter to express that a function assumes ownership of a `widget`
- **R.33** Take a `unique_ptr<widget>&` parameter to express that a function reseats the `widget`
- **R.34** Take a `shared_ptr<widget>` parameter to express shared ownership
- **R.35** Take a `shared_ptr<widget>&` parameter to express that a function might reseat the shared pointer
- **R.36** Take a `const shared_ptr<widget>&` parameter to express that it might retain a reference count to the object ???

Another policy

"If the body of the function is doing work that is comparable to the cost of a move-construction, then do the const T& + T&& overload dance. If the amount of inherent work in the function is already large (or the type is

Winters' rule of thumb

```
// Titus Winters' rule of thumb for a *sunk* parameter:  
//  
// If the work in the function is comparable to a move, do the  
// const& + && overload dance (this is why push_back does).  
void push_back(const std::string& value);  
void push_back(std::string&& value);  
  
// If the work is already large, or the type is cheap to move,  
// just take it by value -- it is much simpler.  
void write_to_disk(std::string path);
```


Winters' rule of thumb

```
// Titus Winters' rule of thumb for a *sunk* parameter:  
//  
// If the work in the function is comparable to a move, do the  
// const& + && overload dance (this is why push_back does).  
void push_back(const std::string& value);  
void push_back(std::string&& value);  
  
// If the work is already large, or the type is cheap to move,  
// just take it by value -- it is much simpler.  
void write_to_disk(std::string path);
```

Winters' rule of thumb

```
// Titus Winters' rule of thumb for a *sunk* parameter:  
//  
// If the work in the function is comparable to a move, do the  
// const& + && overload dance (this is why push_back does).  
void push_back(const std::string& value);  
void push_back(std::string&& value);  
  
// If the work is already large, or the type is cheap to move,  
// just take it by value -- it is much simpler.  
void write_to_disk(std::string path);
```

Overloading

One name, several functions

An overload set is a `*name*`

```
namespace A {  
    int foo(int);  
  
    namespace B {  
        int foo(char);  
        int foo(long);  
  
        // Lookup stops at the first scope that has the name, so this  
        // call only ever considers B's two overloads.  
        int bar(int x) {  
            return foo(static_cast<long>(x));  
        }  
    }  
}
```

An overload set is a **name**

```
namespace A {  
    int foo(int);  
  
    namespace B {  
        int foo(char);  
        int foo(long);  
  
        // Lookup stops at the first scope that has the name, so this  
        // call only ever considers B's two overloads.  
        int bar(int x) {  
            return foo(static_cast<long>(x));  
        }  
    }  
}
```

An overload set is a **name**

```
namespace A {  
    int foo(int);  
  
    namespace B {  
        int foo(char);  
        int foo(long);  
  
        // Lookup stops at the first scope that has the name, so this  
        // call only ever considers B's two overloads.  
        int bar(int x) {  
            return foo(static_cast<long>(x));  
        }  
    }  
}
```


An overload set is a `*name*`

```
namespace A {  
    int foo(int);  
  
    namespace B {  
        int foo(char);  
        int foo(long);  
  
        // Lookup stops at the first scope that has the name, so this  
        // call only ever considers B's two overloads.  
        int bar(int x) {  
            return foo(static_cast<long>(x));  
        }  
    }  
}
```

...gathered by name lookup, including ADL

```
namespace audio {  
    struct Buffer {};  
  
    // Found by argument-dependent lookup, from anywhere.  
    int size(const Buffer&) {  
        return 512;  
    }  
}
```

```
int frames() {
```

...gathered by name lookup, including ADL

```
namespace audio {  
    struct Buffer {};  
  
    // Found by argument-dependent lookup, from anywhere.  
    int size(const Buffer&) {  
        return 512;  
    }  
}
```

```
int frames() {  
    return audio::Buffer::size();  
}
```


...gathered by name lookup, including ADL

```
namespace audio {  
    struct Buffer {};  
  
    // Found by argument-dependent lookup, from anywhere.  
    int size(const Buffer&) {  
        return 512;  
    }  
}
```

```
int frames() {
```

C.138: watch out for hiding

```
struct Base {  
    int gain(int);  
};  
  
struct Derived : Base {  
    // This *hides* Base::gain rather than joining it: Derived{}.gain(1)  
    // calls this one, converting int to double.  
    int gain(double);  
};  
  
struct Fixed : Base {  
    using Base::gain; // C.138: build the overload set explicitly  
    int gain(double);  
};
```

C.138: watch out for hiding

```
struct Base {  
    int gain(int);  
};  
  
struct Derived : Base {  
    // This *hides* Base::gain rather than joining it: Derived{}.gain(1)  
    // calls this one, converting int to double.  
    int gain(double);  
};  
  
struct Fixed : Base {  
    using Base::gain; // C.138: build the overload set explicitly  
    int gain(double);  
};
```


C.138: watch out for hiding

```
struct Base {  
    int gain(int);  
};  
  
struct Derived : Base {  
    // This *hides* Base::gain rather than joining it: Derived{}.gain(1)  
    // calls this one, converting int to double.  
    int gain(double);  
};  
  
struct Fixed : Base {  
    using Base::gain; // C.138: build the overload set explicitly  
    int gain(double);  
};
```

```
// MUST FAIL: the name of an overload set is not a value, so it cannot
// be deduced as a template argument. This is why we wrap overload sets
// in a lambda or a function object.
```

```
int f(int);
double f(double);
```

```
template <class F> void call(F);
```

```
int main() {
    call(f); // error: cannot deduce F from an overload set
}
```

```
// MUST FAIL: the name of an overload set is not a value, so it cannot
// be deduced as a template argument. This is why we wrap overload sets
// in a lambda or a function object.
```

```
int f(int);
double f(double);
```

```
template <class F> void call(F);
```

```
int main() {
    call(f); // error: cannot deduce F from an overload set
}
```



```
// MUST FAIL: the name of an overload set is not a value, so it cannot
// be deduced as a template argument. This is why we wrap overload sets
// in a lambda or a function object.
```

```
int f(int);
double f(double);
```

```
template <class F> void call(F);
```

```
int main() {
    call(f); // error: cannot deduce F from an overload set
}
```

```
// MUST FAIL: the name of an overload set is not a value, so it cannot
// be deduced as a template argument. This is why we wrap overload sets
// in a lambda or a function object.
```

```
int f(int);
double f(double);
```

```
template <class F> void call(F);
```

```
int main() {
    call(f); // error: cannot deduce F from an overload set
}
```

talk hero

1600x900

<https://www.youtube.com/watch?v=OAFFkHqIks0>

Don't Get Overloaded by Overload Sets

Roth Michaels - C++ on Sea 2025



And tomorrow, in this track

- **Back to Basics: Function Overloading** — Chris Ryan
 - <https://cppcon2026.sched.com/event/2RT5f/back-to-basics-function-overloading>
- Go to that one
 - I am going to keep overloading to the minimum I need today

Function types and function pointers

The oldest way to pass a callable

A function has a type

```
// The type of `twice` is "function taking int, returning int".
using unary_int_function = int(int);

static_assert(std::is_same_v<decltype(twice), unary_int_function>);

// You cannot have one of those. You can only have a pointer to one.
static_assert(!std::is_object_v<unary_int_function>);
```

A function has a type

```
// The type of `twice` is "function taking int, returning int".
using unary_int_function = int(int);

static_assert(std::is_same_v<decltype(twice), unary_int_function>);

// You cannot have one of those. You can only have a pointer to one.
static_assert(!std::is_object_v<unary_int_function>);
```


A function has a type

```
// The type of `twice` is "function taking int, returning int".
using unary_int_function = int(int);

static_assert(std::is_same_v<decltype(twice), unary_int_function>);

// You cannot have one of those. You can only have a pointer to one.
static_assert(!std::is_object_v<unary_int_function>);
```

A function has a type

```
// The type of `twice` is "function taking int, returning int".
using unary_int_function = int(int);

static_assert(std::is_same_v<decltype(twice), unary_int_function>);

// You cannot have one of those. You can only have a pointer to one.
static_assert(!std::is_object_v<unary_int_function>);
```

So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```

So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```


So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```

So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```

So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```

So you have a pointer to one

```
using unary_int_function_ptr = int (*)(int);
```

```
// The name of a function decays to a pointer wherever a value is  
// wanted, so all three of these say the same thing.
```

```
unary_int_function_ptr p1 = twice;  
unary_int_function_ptr p2 = &twice;  
unary_int_function_ptr p3 = *twice;
```


...and the call always goes through it

```
// ...and the call goes through the pointer either way.  
static_assert(std::is_same_v<decltype(p1(21)), int>);
```

There are function references too

```
// There are function *references* too. Rarely useful, but they exist,  
// and unlike a pointer one can never be null.  
using unary_int_function_ref = int (&)(int);
```

```
unary_int_function_ref r = twice;
```

There are function references too

```
// There are function *references* too. Rarely useful, but they exist,  
// and unlike a pointer one can never be null.
```

```
using unary_int_function_ref = int (&)(int);
```

```
unary_int_function_ref r = twice;
```

There are function references too

```
// There are function *references* too. Rarely useful, but they exist,  
// and unlike a pointer one can never be null.  
using unary_int_function_ref = int (&)(int);
```

```
unary_int_function_ref r = twice;
```


Handing one to an algorithm

```
bool is_even(int n) {  
    return n % 2 == 0;  
}  
  
int count_even(const std::vector<int>& v) {  
    // count_if takes its predicate by value: here that value is a  
    // pointer, and the call inside the loop is indirect.  
    return static_cast<int>(std::count_if(v.begin(), v.end(), is_even));  
}
```

Handing one to an algorithm

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
int count_even(const std::vector<int>& v) {  
    // count_if takes its predicate by value: here that value is a  
    // pointer, and the call inside the loop is indirect.  
    return static_cast<int>(std::count_if(v.begin(), v.end(), is_even));  
}
```

Handing one to an algorithm

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
int count_even(const std::vector<int>& v) {  
    // count_if takes its predicate by value: here that value is a  
    // pointer, and the call inside the loop is indirect.  
    return static_cast<int>(std::count_if(v.begin(), v.end(), is_even));  
}
```

Handing one to an algorithm

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
int count_even(const std::vector<int>& v) {  
    // count_if takes its predicate by value: here that value is a  
    // pointer, and the call inside the loop is indirect.  
    return static_cast<int>(std::count_if(v.begin(), v.end(), is_even));  
}
```


Picking one out of an overload set

```
int half(int n) {  
    return n / 2;  
}
```

```
double half(double x) {  
    return x / 2.0;  
}
```

```
// `&half` names a set, not a function, so we have to say which one.
```

```
// The target type of the initialisation is enough...
```

```
int (*half_int)(int) = &half;
```

```
// ...or spell the choice out with a cast.
```

```
auto* half_double = static_cast<double (*)>(&half);
```

Picking one out of an overload set

```
int half(int n) {  
    return n / 2;  
}
```

```
double half(double x) {  
    return x / 2.0;  
}
```

```
// `&half` names a set, not a function, so we have to say which one.
```

```
// The target type of the initialisation is enough...
```

```
int (*half_int)(int) = &half;
```

```
// ...or spell the choice out with a cast.
```

```
auto* half_double = static_cast<double (*)>(&half);
```

Picking one out of an overload set

```
int half(int n) {  
    return n / 2;  
}
```

```
double half(double x) {  
    return x / 2.0;  
}
```

```
// `&half` names a set, not a function, so we have to say which one.
```

```
// The target type of the initialisation is enough...
```

```
int (*half_int)(int) = &half;
```

```
// ...or spell the choice out with a cast.
```

```
auto* half_double = static_cast<double (*)>(&half);
```

Picking one out of an overload set

```
int half(int n) {  
    return n / 2;  
}  
double half(double x) {  
    return x / 2.0;  
}  
  
// `&half` names a set, not a function, so we have to say which one.  
// The target type of the initialisation is enough...  
int (*half_int)(int) = &half;  
  
// ...or spell the choice out with a cast.  
auto* half_double = static_cast<double (*)>(&half);
```


Where to park one

```
// Any function pointer converts to any other function pointer type  
// and back again. An object pointer is a different animal: casting a  
// function pointer to void* is not portable, so this -- not void* --  
// is where you park one.
```

```
using any_function = void (*)(*);
```

```
any_function parked = reinterpret_cast<any_function>(half_int);
```

Where to park one

```
// Any function pointer converts to any other function pointer type  
// and back again. An object pointer is a different animal: casting a  
// function pointer to void* is not portable, so this -- not void* --  
// is where you park one.
```

```
using any_function = void (*)(());
```

```
any_function parked = reinterpret_cast<any_function>(half_int);
```

Where to park one

```
// Any function pointer converts to any other function pointer type  
// and back again. An object pointer is a different animal: casting a  
// function pointer to void* is not portable, so this -- not void* --  
// is where you park one.
```

```
using any_function = void (*)(*);
```

```
any_function parked = reinterpret_cast<any_function>(half_int);
```

Where to park one

```
// Any function pointer converts to any other function pointer type  
// and back again. An object pointer is a different animal: casting a  
// function pointer to void* is not portable, so this -- not void* --  
// is where you park one.
```

```
using any_function = void (*)(*);
```

```
any_function parked = reinterpret_cast<any_function>(half_int);
```

Where to park one

```
// Any function pointer converts to any other function pointer type  
// and back again. An object pointer is a different animal: casting a  
// function pointer to void* is not portable, so this -- not void* --  
// is where you park one.  
using any_function = void (*);
```

```
any_function parked = reinterpret_cast<any_function>(half_int);
```


Member functions

An overload set with an invisible parameter

Overloaded on the argument

```
class Track {  
public:  
    void set_gain(float gain) { m_gain = gain; }  
  
    // Overloaded on the argument, like any other function.  
    void set_gain(int percent) { m_gain = static_cast<float>(percent) / 100.f; }  
  
    float gain() const { return m_gain; }  
  
private:  
    float m_gain{1.f};  
};
```

Overloaded on the argument

```
class Track {  
public:  
    void set_gain(float gain) { m_gain = gain; }  
  
    // Overloaded on the argument, like any other function.  
    void set_gain(int percent) { m_gain = static_cast<float>(percent) / 100.f; }  
  
    float gain() const { return m_gain; }  
  
private:  
    float m_gain{1.f};  
};
```

Overloaded on the argument

```
class Track {  
public:  
    void set_gain(float gain) { m_gain = gain; }  
  
    // Overloaded on the argument, like any other function.  
    void set_gain(int percent) { m_gain = static_cast<float>(percent) / 100.f; }  
  
    float gain() const { return m_gain; }  
  
private:  
    float m_gain{1.f};  
};
```

Overloaded on the *object*

```
class DataHolder {  
public:  
    // Overloaded on the value category of the object itself.  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:
```


Overloaded on the *object*

```
class DataHolder {  
public:  
    // Overloaded on the value category of the object itself.  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:
```

Overloaded on the *object*

```
class DataHolder {  
public:  
    // Overloaded on the value category of the object itself.  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:
```

Overloaded on the *object*

```
class DataHolder {  
public:  
    // Overloaded on the value category of the object itself.  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:
```

The parameter you did not write: `this`

```
class Counter {
public:
    // What you write...
    void bump() { ++m_count; }

    // ...is what you would write by hand if the object were spelled
    // out. `this` is the implicit object parameter.
    void bump_by_hand() { ++this->m_count; }

    int count() const { return m_count; }

private:
    int m_count{};
};

// A free function has to say it out loud, and the call site changes.
void bump(Counter* self);
```

The parameter you did not write: `this`

```
class Counter {
public:
    // What you write...
    void bump() { ++m_count; }

    // ...is what you would write by hand if the object were spelled
    // out. `this` is the implicit object parameter.
    void bump_by_hand() { ++this->m_count; }

    int count() const { return m_count; }

private:
    int m_count{};
};

// A free function has to say it out loud, and the call site changes.
void bump(Counter* self);
```


The parameter you did not write: `this`

```
class Counter {
public:
    // What you write...
    void bump() { ++m_count; }

    // ...is what you would write by hand if the object were spelled
    // out. `this` is the implicit object parameter.
    void bump_by_hand() { ++this->m_count; }

    int count() const { return m_count; }

private:
    int m_count{};
};

// A free function has to say it out loud, and the call site changes.
void bump(Counter* self);
```

The parameter you did not write: `this`

```
class Counter {
public:
    // What you write...
    void bump() { ++m_count; }

    // ...is what you would write by hand if the object were spelled
    // out. `this` is the implicit object parameter.
    void bump_by_hand() { ++this->m_count; }

    int count() const { return m_count; }

private:
    int m_count{};
};

// A free function has to say it out loud, and the call site changes.
void bump(Counter* self);
```

C++23: say it out loud (P0847)

```
class Buffer {  
public:  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:  
    std::vector<float> m_data;  
};
```

C++23: say it out loud (P0847)

```
class Buffer {  
public:  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:  
    std::vector<float> m_data;  
};
```

C++23: say it out loud (P0847)

```
class Buffer {  
public:  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:  
    std::vector<float> m_data;  
};
```


C++23: say it out loud (P0847)

```
class Buffer {  
public:  
    std::vector<float>& data() & { return m_data; }  
  
    const std::vector<float>& data() const& { return m_data; }  
  
    std::vector<float>&& data() && { return std::move(m_data); }  
  
private:  
    std::vector<float> m_data;  
};
```

```
class DeducedBuffer {
public:
    // `Self` deduces the const-ness *and* the value category of the
    // object, so forwarding it forwards the member.
    template <class Self> auto&& data(this Self&& self) { return std::forward<Self>(self).m_data; }

private:
    std::vector<float> m_data;
};
```

```
class DeducedBuffer {
public:
    // `Self` deduces the const-ness *and* the value category of the
    // object, so forwarding it forwards the member.
    template <class Self> auto&& data(this Self&& self) { return std::forward<Self>(self).m_data; }

private:
    std::vector<float> m_data;
};
```

It also kills CRTP

```
// The other thing it buys you: mixins without CRTP.
```

```
struct AddPostfix {  
    auto operator++(this auto& self, int) {  
        auto copy = self;  
        ++self;  
        return copy;  
    }  
};
```

```
struct Frame : AddPostfix {  
    int value{};  
  
    Frame& operator++() {  
        ++value;  
        return *this;  
    }  
};
```

It also kills CRTP

```
// The other thing it buys you: mixins without CRTP.
struct AddPostfix {
    auto operator++(this auto& self, int) {
        auto copy = self;
        ++self;
        return copy;
    }
};

struct Frame : AddPostfix {
    int value{};

    Frame& operator++() {
        ++value;
        return *this;
    }
};
```


It also kills CRTP

```
// The other thing it buys you: mixins without CRTP.
struct AddPostfix {
    auto operator++(this auto& self, int) {
        auto copy = self;
        ++self;
        return copy;
    }
};

struct Frame : AddPostfix {
    int value{};

    Frame& operator++() {
        ++value;
        return *this;
    }
};
```

It also kills CRTP

```
// The other thing it buys you: mixins without CRTP.
struct AddPostfix {
    auto operator++(this auto& self, int) {
        auto copy = self;
        ++self;
        return copy;
    }
};

struct Frame : AddPostfix {
    int value{};

    Frame& operator++() {
        ++value;
        return *this;
    }
};
```

A pointer to a member function

```
struct Track {  
    std::string name;  
    bool muted{false};  
  
    bool is_audible() const { return !muted; }  
};
```

```
// Note where the class name goes: it is part of the type.  
using audible_query = bool (Track::*)() const;
```

A pointer to a member function

```
struct Track {  
    std::string name;  
    bool muted{false};  
  
    bool is_audible() const { return !muted; }  
};
```

```
// Note where the class name goes: it is part of the type.  
using audible_query = bool (Track::*)() const;
```

A pointer to a member function

```
struct Track {  
    std::string name;  
    bool muted{false};  
  
    bool is_audible() const { return !muted; }  
};
```

```
// Note where the class name goes: it is part of the type.  
using audible_query = bool (Track::* )() const;
```


Calling through one

```
bool ask(const Track& t, audible_query q) {  
    // .* for an object, →* for a pointer. The parentheses are not  
    // optional: .* binds looser than the call.  
    return (t.*q)();  
}
```

```
bool ask(const Track* t, audible_query q) {  
    return (t→*q)();  
}
```

Calling through one

```
bool ask(const Track& t, audible_query q) {  
    // .* for an object, →* for a pointer. The parentheses are not  
    // optional: .* binds looser than the call.  
    return (t.*q)();  
}
```

```
bool ask(const Track* t, audible_query q) {  
    return (t→*q)();  
}
```

Calling through one

```
bool ask(const Track& t, audible_query q) {  
    // .* for an object, →* for a pointer. The parentheses are not  
    // optional: .* binds looser than the call.  
    return (t.*q)();  
}
```

```
bool ask(const Track* t, audible_query q) {  
    return (t→*q)();  
}
```

`std::invoke` papers over all of it

```
// std::invoke papers over all of it: member functions, data members,  
// function pointers and function objects are all just "callable".  
bool ask_invoke(const Track& t, audible_query q) {  
    return std::invoke(q, t);  
}  
  
// A pointer to a *data* member is callable too.  
std::string name_of(const Track& t) {  
    return std::invoke(&Track::name, t);  
}
```

`std::invoke` papers over all of it

```
// std::invoke papers over all of it: member functions, data members,  
// function pointers and function objects are all just "callable".  
bool ask_invoke(const Track& t, audible_query q) {  
    return std::invoke(q, t);  
}  
  
// A pointer to a *data* member is callable too.  
std::string name_of(const Track& t) {  
    return std::invoke(&Track::name, t);  
}
```


`std::invoke` papers over all of it

```
// std::invoke papers over all of it: member functions, data members,  
// function pointers and function objects are all just "callable".  
bool ask_invoke(const Track& t, audible_query q) {  
    return std::invoke(q, t);  
}  
  
// A pointer to a *data* member is callable too.  
std::string name_of(const Track& t) {  
    return std::invoke(&Track::name, t);  
}
```



```
// mem_fn wraps one up as a function object an algorithm will accept.
std::ptrdiff_t count_audible(const std::vector<Track>& tracks) {
    return std::count_if(tracks.begin(), tracks.end(), std::mem_fn(&Track::is_audible));
}
```

```
// mem_fn wraps one up as a function object an algorithm will accept.
std::ptrdiff_t count_audible(const std::vector<Track>& tracks) {
    return std::count_if(tracks.begin(), tracks.end(), std::mem_fn(&Track::is_audible));
}
```

operator()

Function objects

"Functor" is an overloaded word

- **Category theory** — a structure-preserving map between categories
 - Haskell's Functor: $\text{fmap} :: (a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$
- **ML / OCaml** — a *module* parameterised on another module
 - what C++ calls a class template, roughly
- **Prolog, term rewriting** — the symbol at the head of a term
- **C++** — "a thing you can call () on"

You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-(Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```


You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-(Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```


You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-(Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```

You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-(Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```

You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```

You already overload operators

```
struct Point {  
    float x{};  
    float y{};  
  
    friend constexpr bool operator==(Point, Point) = default;  
};  
  
constexpr Point operator+(Point a, Point b) {  
    return {a.x + b.x, a.y + b.y};  
}  
  
constexpr Point operator-(Point operator-p {  
    return {-p.x, -p.y};  
}  
  
constexpr Point operator*(Point p, float scale) {  
    return {p.x * scale, p.y * scale};  
}
```

...including the weird ones

```
struct Grid {  
    Point origin{};  
    float spacing{1.f};  
  
    // operator[] is fair game too.  
    constexpr Point operator[](int n) const {  
        return origin + Point{spacing, 0.f} * static_cast<float>(n);  
    }  
};
```


...including the weird ones

```
struct Grid {  
    Point origin{};  
    float spacing{1.f};  
  
    // operator[] is fair game too.  
    constexpr Point operator[](int n) const {  
        return origin + Point{spacing, 0.f} * static_cast<float>(n);  
    }  
};
```


So: `operator()`

```
// ...and so is operator(). Nothing about it is special: it is an
// operator you can overload, and that is the whole trick.
struct Translate {
    Point offset{};

    constexpr Point operator()(Point p) const { return p + offset; }
};
```

So: `operator()`

```
// ...and so is operator(). Nothing about it is special: it is an
// operator you can overload, and that is the whole trick.
struct Translate {
    Point offset{};

    constexpr Point operator()(Point p) const { return p + offset; }
};
```

So: `operator()`

```
// ...and so is operator(). Nothing about it is special: it is an
// operator you can overload, and that is the whole trick.
struct Translate {
    Point offset{};

    constexpr Point operator()(Point p) const { return p + offset; }
};
```

A function object can carry state

```
class Gain {  
public:  
    explicit constexpr Gain(float gain) : m_gain{gain} {}  
  
    constexpr float operator()(float sample) const { return sample * m_gain; }  
  
private:  
    float m_gain;  
};
```

A function object can carry state

```
class Gain {  
public:  
    explicit constexpr Gain(float gain) : m_gain{gain} {}  
  
    constexpr float operator()(float sample) const { return sample * m_gain; }  
  
private:  
    float m_gain;  
};
```


A function object can carry state

```
class Gain {  
public:  
    explicit constexpr Gain(float gain) : m_gain{gain} {}  
  
    constexpr float operator()(float sample) const { return sample * m_gain; }  
  
private:  
    float m_gain;  
};
```


A function object can carry state

```
class Gain {  
public:  
    explicit constexpr Gain(float gain) : m_gain{gain} {}  
  
    constexpr float operator()(float sample) const { return sample * m_gain; }  
  
private:  
    float m_gain;  
};
```

...and it can overload the call

```
class Fader {  
public:  
    explicit constexpr Fader(float gain) : m_gain{gain} {}  
  
    // operator() is an overload set like any other member function.  
    constexpr float operator()(float sample) const { return sample * m_gain; }  
  
    void operator()(std::span<float> block) const {  
        for (auto& sample : block) {  
            sample *= m_gain;  
        }  
    }  
}  
  
private:
```

...and it can overload the call

```
class Fader {
public:
    explicit constexpr Fader(float gain) : m_gain{gain} {}

    // operator() is an overload set like any other member function.
    constexpr float operator()(float sample) const { return sample * m_gain; }

    void operator()(std::span<float> block) const {
        for (auto& sample : block) {
            sample *= m_gain;
        }
    }
}

private:
```

...and it can overload the call

```
class Fader {
public:
    explicit constexpr Fader(float gain) : m_gain{gain} {}

    // operator() is an overload set like any other member function.
    constexpr float operator()(float sample) const { return sample * m_gain; }

    void operator()(std::span<float> block) const {
        for (auto& sample : block) {
            sample *= m_gain;
        }
    }
}

private:
```

...and it can be a template

```
// It can be a template, which is how std::less< > and friends manage
// to be "transparent".
struct Negate {
    template <class T> constexpr T operator()(T x) const { return -x; }
};
```


...and it can be a template

```
// It can be a template, which is how std::less<> and friends manage
// to be "transparent".
struct Negate {
    template <class T> constexpr T operator()(T x) const { return -x; }
};
```


...and it does not have to be `const`

```
// And it does not have to be const. This one answers differently
// every time you call it -- which is exactly what a PRNG does.
class Counter {
public:
    constexpr int operator()() { return ++m_count; }

private:
    int m_count{};
};
```

...and it does not have to be `const`

```
// And it does not have to be const. This one answers differently
// every time you call it -- which is exactly what a PRNG does.
class Counter {
public:
    constexpr int operator()() { return ++m_count; }

private:
    int m_count{};
};
```

C++23: `static operator()` (P1169)

```
struct is_even {  
    // No captures, no state: there is nothing for an implicit object  
    // parameter to point at, so C++23 lets us drop it.  
    static bool operator()(int n) { return n % 2 == 0; }  
};
```

C++23: `static operator()` (P1169)

```
struct is_even {  
    // No captures, no state: there is nothing for an implicit object  
    // parameter to point at, so C++23 lets us drop it.  
    static bool operator()(int n) { return n % 2 == 0; }  
};
```

C++23: `static operator()` (P1169)

```
struct is_even {  
    // No captures, no state: there is nothing for an implicit object  
    // parameter to point at, so C++23 lets us drop it.  
    static bool operator()(int n) { return n % 2 == 0; }  
};
```


The standard library has been doing this since 1998

```
void sort_descending(std::vector<int>& v) {  
    // std::greater<int>{} is an object. The {} constructs it; the  
    // algorithm then calls it.  
    std::sort(v.begin(), v.end(), std::greater<int>{});  
}  
  
int total(const std::vector<int>& v) {  
    return std::accumulate(v.begin(), v.end(), 0, std::plus<int>{});  
}
```

The standard library has been doing this since 1998

```
void sort_descending(std::vector<int>& v) {  
    // std::greater<int>{} is an object. The {} constructs it; the  
    // algorithm then calls it.  
    std::sort(v.begin(), v.end(), std::greater<int>{});  
}  
  
int total(const std::vector<int>& v) {  
    return std::accumulate(v.begin(), v.end(), 0, std::plus<int>{});  
}
```


The standard library has been doing this since 1998

```
void sort_descending(std::vector<int>& v) {  
    // std::greater<int>{} is an object. The {} constructs it; the  
    // algorithm then calls it.  
    std::sort(v.begin(), v.end(), std::greater<int>{});  
}  
  
int total(const std::vector<int>& v) {  
    return std::accumulate(v.begin(), v.end(), 0, std::plus<int>{});  
}
```

The ``<>`` ones are transparent

```
// The <> versions have a *templated* operator(), so one object serves
// every type -- and comparing avoids building a temporary key.
using ByName = std::set<std::string, std::less<>>;

bool contains(const ByName& names, std::string_view name) {
    return names.find(name) != names.end();
}
```

The ``<>`` ones are transparent

```
// The <> versions have a *templated* operator(), so one object serves
// every type -- and comparing avoids building a temporary key.
using ByName = std::set<std::string, std::less<>>;

bool contains(const ByName& names, std::string_view name) {
    return names.find(name) != names.end();
}
```

The ``<>`` ones are transparent

```
// The <> versions have a *templated* operator(), so one object serves
// every type -- and comparing avoids building a temporary key.
using ByName = std::set<std::string, std::less<>>;
```

```
bool contains(const ByName& names, std::string_view name) {
    return names.find(name)  $\neq$  names.end();
}
```

...and some of them make new ones

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
std::ptrdiff_t count_odd(const std::vector<int>& v) {  
    // not_fn wraps a callable and returns a function object.  
    return std::count_if(v.begin(), v.end(), std::not_fn(is_even));  
}
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
// bind_front bakes in the leading arguments and hands back an object.
```


...and some of them make new ones

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
std::ptrdiff_t count_odd(const std::vector<int>& v) {  
    // not_fn wraps a callable and returns a function object.  
    return std::count_if(v.begin(), v.end(), std::not_fn(is_even));  
}
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
// bind_front bakes in the leading arguments and hands back an object.
```

...and some of them make new ones

```
bool is_even(int n) {  
    return n % 2 == 0;  
}
```

```
std::ptrdiff_t count_odd(const std::vector<int>& v) {  
    // not_fn wraps a callable and returns a function object.  
    return std::count_if(v.begin(), v.end(), std::not_fn(is_even));  
}
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
// bind_front bakes in the leading arguments and hands back an object.
```


`<random>` is nothing but function objects

```
// <random> is entirely built out of function objects: an engine is
// called with no arguments, a distribution is called with an engine.
float one_sample(std::mt19937& engine) {
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};

    const auto raw = engine(); // engine's operator()
    (void)raw;

    return dist(engine); // distribution's operator()
}
```

`<random>` is nothing but function objects

```
// <random> is entirely built out of function objects: an engine is
// called with no arguments, a distribution is called with an engine.
float one_sample(std::mt19937& engine) {
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};

    const auto raw = engine(); // engine's operator()
    (void)raw;

    return dist(engine); // distribution's operator()
}
```

`<random>` is nothing but function objects

```
// <random> is entirely built out of function objects: an engine is
// called with no arguments, a distribution is called with an engine.
float one_sample(std::mt19937& engine) {
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};

    const auto raw = engine(); // engine's operator()
    (void)raw;

    return dist(engine); // distribution's operator()
}
```

A function object from a real plug-in

```
template <class PRNG = std::mt19937> class UniformWhiteNoise {
public:
    using SeedType = typename PRNG::result_type;

    explicit UniformWhiteNoise(SeedType seed) : m_gen{seed} {}

    // The whole interface: call it, get the next sample.
    float operator()() { return m_gain * m_dist(m_gen); }

    void SetGain(float gain) { m_gain = gain; }

private:
    PRNG m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
```

A function object from a real plug-in

```
template <class PRNG = std::mt19937> class UniformWhiteNoise {
public:
    using SeedType = typename PRNG::result_type;

    explicit UniformWhiteNoise(SeedType seed) : m_gen{seed} {}

    // The whole interface: call it, get the next sample.
    float operator()() { return m_gain * m_dist(m_gen); }

    void SetGain(float gain) { m_gain = gain; }

private:
    PRNG m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
}
```


A function object from a real plug-in

```
template <class PRNG = std::mt19937> class UniformWhiteNoise {
public:
    using SeedType = typename PRNG::result_type;

    explicit UniformWhiteNoise(SeedType seed) : m_gen{seed} {}

    // The whole interface: call it, get the next sample.
    float operator()() { return m_gain * m_dist(m_gen); }

    void SetGain(float gain) { m_gain = gain; }

private:
    PRNG m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
}
```

A function object from a real plug-in

```
template <class PRNG = std::mt19937> class UniformWhiteNoise {
public:
    using SeedType = typename PRNG::result_type;

    explicit UniformWhiteNoise(SeedType seed) : m_gen{seed} {}

    // The whole interface: call it, get the next sample.
    float operator()() { return m_gain * m_dist(m_gen); }

    void SetGain(float gain) { m_gain = gain; }

private:
    PRNG m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
}
```


A function object from a real plug-in

```
template <class PRNG = std::mt19937> class UniformWhiteNoise {
public:
    using SeedType = typename PRNG::result_type;

    explicit UniformWhiteNoise(SeedType seed) : m_gen{seed} {}

    // The whole interface: call it, get the next sample.
    float operator()() { return m_gain * m_dist(m_gen); }

    void SetGain(float gain) { m_gain = gain; }

private:
    PRNG m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
};
```

...and `std::generate` just calls it

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto buffer = std::vector<float>(frames);  
  
    // std::generate calls it once per sample. The engine state lives  
    // in the object, so no globals and no static -- and two of these  
    // with different seeds are independent.  
    std::generate(buffer.begin(), buffer.end(), UniformWhiteNoise{seed});  
  
    return buffer;  
}
```

...and `std::generate` just calls it

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto buffer = std::vector<float>(frames);  
  
    // std::generate calls it once per sample. The engine state lives  
    // in the object, so no globals and no static -- and two of these  
    // with different seeds are independent.  
    std::generate(buffer.begin(), buffer.end(), UniformWhiteNoise{seed});  
  
    return buffer;  
}
```

...and `std::generate` just calls it

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto buffer = std::vector<float>(frames);  
  
    // std::generate calls it once per sample. The engine state lives  
    // in the object, so no globals and no static -- and two of these  
    // with different seeds are independent.  
    std::generate(buffer.begin(), buffer.end(), UniformWhiteNoise{seed});  
  
    return buffer;  
}
```

Careful: algorithms take callables *by value*

```
// Careful: generate takes the generator *by value*. Pass an lvalue
// and the engine that advances is the copy, not yours.
std::vector<float> two_buffers_the_same(std::size_t frames, unsigned seed) {
    auto noise = UniformWhiteNoise<>{seed};
    auto first = std::vector<float>(frames);
    auto second = std::vector<float>(frames);

    std::generate(first.begin(), first.end(), noise);
    std::generate(second.begin(), second.end(), noise);

    assert(first == second); // ...surprise
    return first;
}
```


Careful: algorithms take callables *by value*

```
// Careful: generate takes the generator *by value*. Pass an lvalue
// and the engine that advances is the copy, not yours.
std::vector<float> two_buffers_the_same(std::size_t frames, unsigned seed) {
    auto noise = UniformWhiteNoise<>{seed};
    auto first = std::vector<float>(frames);
    auto second = std::vector<float>(frames);

    std::generate(first.begin(), first.end(), noise);
    std::generate(second.begin(), second.end(), noise);

    assert(first == second); // ...surprise
    return first;
}
```

Careful: algorithms take callables *by value*

```
// Careful: generate takes the generator *by value*. Pass an lvalue
// and the engine that advances is the copy, not yours.
std::vector<float> two_buffers_the_same(std::size_t frames, unsigned seed) {
    auto noise = UniformWhiteNoise<>{seed};
    auto first = std::vector<float>(frames);
    auto second = std::vector<float>(frames);

    std::generate(first.begin(), first.end(), noise);
    std::generate(second.begin(), second.end(), noise);

    assert(first == second); // ...surprise
    return first;
}
```


Visiting a `std::variant`

The other place function objects earn their keep

A visitor is just a function object

```
struct Describe {  
    std::string operator()(int i) const { return "int: " + std::to_string(i); }  
  
    std::string operator()(const std::string& s) const { return "string: " + s; }  
  
    std::string operator()(std::optional<int> o) const {  
        return o ? "optional: " + std::to_string(*o) : "optional: empty";  
    }  
};  
  
std::string describe(const MyThings& thing) {  
    return std::visit(Describe{}, thing);  
}
```

A visitor is just a function object

```
struct Describe {  
    std::string operator()(int i) const { return "int: " + std::to_string(i); }  
  
    std::string operator()(const std::string& s) const { return "string: " + s; }  
  
    std::string operator()(std::optional<int> o) const {  
        return o ? "optional: " + std::to_string(*o) : "optional: empty";  
    }  
};  
  
std::string describe(const MyThings& thing) {  
    return std::visit(Describe{}, thing);  
}
```

A visitor is just a function object

```
struct Describe {  
    std::string operator()(int i) const { return "int: " + std::to_string(i); }  
  
    std::string operator()(const std::string& s) const { return "string: " + s; }  
  
    std::string operator()(std::optional<int> o) const {  
        return o ? "optional: " + std::to_string(*o) : "optional: empty";  
    }  
};  
  
std::string describe(const MyThings& thing) {  
    return std::visit(Describe{}, thing);  
}
```

A visitor is just a function object

```
struct Describe {  
    std::string operator()(int i) const { return "int: " + std::to_string(i); }  
  
    std::string operator()(const std::string& s) const { return "string: " + s; }  
  
    std::string operator()(std::optional<int> o) const {  
        return o ? "optional: " + std::to_string(*o) : "optional: empty";  
    }  
};  
  
std::string describe(const MyThings& thing) {  
    return std::visit(Describe{}, thing);  
}
```


A visitor is just a function object

```
struct Describe {  
    std::string operator()(int i) const { return "int: " + std::to_string(i); }  
  
    std::string operator()(const std::string& s) const { return "string: " + s; }  
  
    std::string operator()(std::optional<int> o) const {  
        return o ? "optional: " + std::to_string(*o) : "optional: empty";  
    }  
};  
  
std::string describe(const MyThings& thing) {  
    return std::visit(Describe{}, thing);  
}
```

The overload set is where exhaustiveness lives

```
// Delete the std::string overload and this stops compiling. That is  
// the point: the visitor is where exhaustiveness gets checked.  
static_assert(std::is_invocable_v<Describe, int>);  
static_assert(std::is_invocable_v<Describe, std::string>);  
static_assert(std::is_invocable_v<Describe, std::optional<int>>);
```


The overload set is where exhaustiveness lives

```
// Delete the std::string overload and this stops compiling. That is  
// the point: the visitor is where exhaustiveness gets checked.  
static_assert(std::is_invocable_v<Describe, int>);  
static_assert(std::is_invocable_v<Describe, std::string>);  
static_assert(std::is_invocable_v<Describe, std::optional<int>>);
```

Lambdas

Function objects, written by the compiler

The smallest lambda

```
// The smallest lambda there is: captures nothing, takes nothing,  
// returns nothing. Note that the parameter list is optional too.  
auto nothing = [] {};
```

The smallest lambda

```
// The smallest lambda there is: captures nothing, takes nothing,  
// returns nothing. Note that the parameter list is optional too.  
auto nothing = [] {};
```

The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```


The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```

The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```


The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```

The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```

The shapes you will actually write

```
auto twice = [](int n) { return n * 2; };           // deduced return
auto half = [](int n) → double { return n / 2.0; }; // trailing return
auto pure = [](int n) constexpr { return n + 1; };  // constexpr
auto safe = [](int n) noexcept { return n; };       // noexcept
auto vararg = [](auto... xs) { return (xs + ... + 0); };
```

All of it, in grammar order

```
// The full declarator, in order:  
//  
//  [captures] <tparams> (params) specifiers noexcept  
//           → return requires { body }  
auto everything = []<class T>(T x) constexpr noexcept → T  
    requires std::integral<T>  
{ return x + 1; };
```

All of it, in grammar order

```
// The full declarator, in order:  
//  
//  [captures] <tparams> (params) specifiers noexcept  
//           → return requires { body }  
auto everything = []<class T>(T x) constexpr noexcept → T  
    requires std::integral<T>  
{ return x + 1; };
```


All of it, in grammar order

```
// The full declarator, in order:  
//  
// [captures] <tparams> (params) specifiers noexcept  
//      → return requires { body }  
auto everything = []<class T>(T x) constexpr noexcept → T  
    requires std::integral<T>  
{ return x + 1; };
```

All of it, in grammar order

```
// The full declarator, in order:  
//  
// [captures] <tparams> (params) specifiers noexcept  
//      → return requires { body }  
auto everything = []<class T>(T x) constexpr noexcept → T  
    requires std::integral<T>  
{ return x + 1; };
```


ES.28: the immediately-invoked lambda

```
// ES.28: a lambda called immediately, to initialise something const
// that needs more than one statement to compute.
const int frames = [] {
    int total = 0;
    for (int block = 0; block < 4; ++block) {
        total += 512;
    }
    return total;
}();
```

ES.28: the immediately-invoked lambda

```
// ES.28: a lambda called immediately, to initialise something const
// that needs more than one statement to compute.
const int frames = [] {
    int total = 0;
    for (int block = 0; block < 4; ++block) {
        total += 512;
    }
    return total;
}();
```

ES.28: the immediately-invoked lambda

```
// ES.28: a lambda called immediately, to initialise something const
// that needs more than one statement to compute.
const int frames = [] {
    int total = 0;
    for (int block = 0; block < 4; ++block) {
        total += 512;
    }
    return total;
}();
```

A lambda is a class

```
class add_x {  
public:  
    explicit add_x(int x) : m_x{x} {}  
  
    // The captures became members...  
    // ...and the body became a const operator().  
    int operator()(int n) const { return n + m_x; }  
  
private:  
    int m_x;  
};
```

A lambda is a class

```
class add_x {  
public:  
    explicit add_x(int x) : m_x{x} {}  
  
    // The captures became members...  
    // ...and the body became a const operator().  
    int operator()(int n) const { return n + m_x; }  
  
private:  
    int m_x;  
};
```


A lambda is a class

```
class add_x {  
public:  
    explicit add_x(int x) : m_x{x} {}  
  
    // The captures became members...  
    // ...and the body became a const operator().  
    int operator()(int n) const { return n + m_x; }  
  
private:  
    int m_x;  
};
```

A lambda is a class

```
class add_x {  
public:  
    explicit add_x(int x) : m_x{x} {}  
  
    // The captures became members...  
    // ...and the body became a const operator().  
    int operator()(int n) const { return n + m_x; }  
  
private:  
    int m_x;  
};
```

A lambda is a class

```
class add_x {  
public:  
    explicit add_x(int x) : m_x{x} {}  
  
    // The captures became members...  
    // ...and the body became a const operator().  
    int operator()(int n) const { return n + m_x; }  
  
private:  
    int m_x;  
};
```


...and this is the same class

```
auto make_add_x(int x) {  
    return [x](int n) { return n + x; };  
}
```

...and this is the same class

```
auto make_add_x(int x) {  
    return [x](int n) { return n + x; };  
}
```

```
// MUST FAIL: a by-copy capture is const inside a non-mutable lambda.  
// If this ever compiles, a claim on the slides has become false.
```

```
int main() {  
    int x = 0;  
  
    auto f = [x]() { x = 1; }; // error: cannot assign to a captured copy  
  
    f();  
}
```

```
// MUST FAIL: a by-copy capture is const inside a non-mutable lambda.  
// If this ever compiles, a claim on the slides has become false.
```

```
int main() {  
    int x = 0;  
  
    auto f = [x]() { x = 1; }; // error: cannot assign to a captured copy  
  
    f();  
}
```

Demo time

<https://cppinsights.io>

What the compiler writes that you cannot

```
// What the compiler writes that you cannot: the closure type has no
// name, no default constructor before C++20, and its copy and move
// operations are whatever the captures' are.
static_assert(!std::is_default_constructible_v<decltype(make_add_x(0))>);
static_assert(std::is_copy_constructible_v<decltype(make_add_x(0))>);

// A lambda with no captures does get one extra trick: it converts to
// a plain function pointer, because there is no state to carry.
auto stateless = [](int n) { return n * 2; };
int (*as_pointer)(int) = stateless;
```

What the compiler writes that you cannot

```
// What the compiler writes that you cannot: the closure type has no
// name, no default constructor before C++20, and its copy and move
// operations are whatever the captures' are.
static_assert(!std::is_default_constructible_v<decltype(make_add_x(0))>);
static_assert(std::is_copy_constructible_v<decltype(make_add_x(0))>);

// A lambda with no captures does get one extra trick: it converts to
// a plain function pointer, because there is no state to carry.
auto stateless = [](int n) { return n * 2; };
int (*as_pointer)(int) = stateless;
```

What the compiler writes that you cannot

```
// What the compiler writes that you cannot: the closure type has no
// name, no default constructor before C++20, and its copy and move
// operations are whatever the captures' are.
static_assert(!std::is_default_constructible_v<decltype(make_add_x(0))>);
static_assert(std::is_copy_constructible_v<decltype(make_add_x(0))>);

// A lambda with no captures does get one extra trick: it converts to
// a plain function pointer, because there is no state to carry.
auto stateless = [](int n) { return n * 2; };
int (*as_pointer)(int) = stateless;
```


What the compiler writes that you cannot

```
// What the compiler writes that you cannot: the closure type has no
// name, no default constructor before C++20, and its copy and move
// operations are whatever the captures' are.
static_assert(!std::is_default_constructible_v<decltype(make_add_x(0))>);
static_assert(std::is_copy_constructible_v<decltype(make_add_x(0))>);

// A lambda with no captures does get one extra trick: it converts to
// a plain function pointer, because there is no state to carry.
auto stateless = [](int n) { return n * 2; };
int (*as_pointer)(int) = stateless;
```

```
// MUST FAIL: only a capture-less lambda converts to a function  
// pointer. There is nowhere in a bare pointer to keep the state.
```

```
int main() {  
    int x = 40;  
  
    auto add_x = [x](int n) { return n + x; };  
  
    int (*p)(int) = add_x; // error: no viable conversion  
  
    return p(2);  
}
```

```
// MUST FAIL: only a capture-less lambda converts to a function  
// pointer. There is nowhere in a bare pointer to keep the state.
```

```
int main() {  
    int x = 40;  
  
    auto add_x = [x](int n) { return n + x; };  
  
    int (*p)(int) = add_x; // error: no viable conversion  
  
    return p(2);  
}
```

```
// MUST FAIL: only a capture-less lambda converts to a function  
// pointer. There is nowhere in a bare pointer to keep the state.
```

```
int main() {  
    int x = 40;  
  
    auto add_x = [x](int n) { return n + x; };  
  
    int (*p)(int) = add_x; // error: no viable conversion  
  
    return p(2);  
}
```

Captures, carefully

Every capture is a member


```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 90);
    h();
    assert(x == 4);
}

```



```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 90);
    h();
    assert(x == 4);
}

```

```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```



```

void every_form() {
    int x = 1;
    auto big = std::string{"expensive"};

    auto a = [] {}; // nothing
    auto b = [x] { return x; }; // a copy of x
    auto c = [&x] { ++x; }; // a reference to x
    auto d = [=] { return x; }; // copy of everything used
    auto e = [&] { ++x; }; // reference to everything used
    auto f = [n = x * 2] { return n; }; // init capture: a new member
    auto g = [s = std::move(big)] { return s.size(); }; // move in
    auto h = [&r = x] { ++r; }; // reference with a new name

    a();
    assert(b() == 1);
    c();
    assert(d() == 1);
    e();
    assert(f() == 2);
    assert(g() == 9u);
    h();
    assert(x == 4);
}

```

`mutable` drops the `const`, nothing more

```
void mutable_lambda() {  
    // A by-copy capture is const inside the body. `mutable` drops the  
    // const from operator(), it does not make the capture a reference.  
    auto next = [n = 0]() mutable { return ++n; };  
  
    assert(next() == 1);  
    assert(next() == 2);  
  
    // The state is in the object, so a copy has its own.  
    auto other = next;  
    assert(other() == 3);  
    assert(next() == 3);  
}
```

`mutable` drops the `const`, nothing more

```
void mutable_lambda() {  
    // A by-copy capture is const inside the body. `mutable` drops the  
    // const from operator(), it does not make the capture a reference.  
    auto next = [n = 0]() mutable { return ++n; };  
  
    assert(next() == 1);  
    assert(next() == 2);  
  
    // The state is in the object, so a copy has its own.  
    auto other = next;  
    assert(other() == 3);  
    assert(next() == 3);  
}
```


`mutable` drops the `const`, nothing more

```
void mutable_lambda() {  
    // A by-copy capture is const inside the body. `mutable` drops the  
    // const from operator(), it does not make the capture a reference.  
    auto next = [n = 0]() mutable { return ++n; };  
  
    assert(next() == 1);  
    assert(next() == 2);  
  
    // The state is in the object, so a copy has its own.  
    auto other = next;  
    assert(other() == 3);  
    assert(next() == 3);  
}
```

`mutable` drops the `const`, nothing more

```
void mutable_lambda() {  
    // A by-copy capture is const inside the body. `mutable` drops the  
    // const from operator(), it does not make the capture a reference.  
    auto next = [n = 0]() mutable { return ++n; };  
  
    assert(next() == 1);  
    assert(next() == 2);  
  
    // The state is in the object, so a copy has its own.  
    auto other = next;  
    assert(other() == 3);  
    assert(next() == 3);  
}
```

```

class Fader {
public:
    // F.54: in a member function a `[=]` default does NOT copy the
    // members. It copies the invisible `this` pointer, so every member
    // access is through a reference to an object you may not own any
    // more. C++20 deprecated the silent version -- say what you mean:
    auto explicit_this() const {
        return [=, this] { return m_gain; };
    }

    // ...or take the whole object with you.
    auto owning() const {
        return [*this] { return m_gain; };
    }

private:
    float m_gain{0.5f};
};

```

```
class Fader {
public:
    // F.54: in a member function a `[=]` default does NOT copy the
    // members. It copies the invisible `this` pointer, so every member
    // access is through a reference to an object you may not own any
    // more. C++20 deprecated the silent version -- say what you mean:
    auto explicit_this() const {
        return [=, this] { return m_gain; };
    }

    // ...or take the whole object with you.
    auto owning() const {
        return [*this] { return m_gain; };
    }

private:
    float m_gain{0.5f};
};
```

```

class Fader {
public:
    // F.54: in a member function a `[=]` default does NOT copy the
    // members. It copies the invisible `this` pointer, so every member
    // access is through a reference to an object you may not own any
    // more. C++20 deprecated the silent version -- say what you mean:
    auto explicit_this() const {
        return [=, this] { return m_gain; };
    }

    // ...or take the whole object with you.
    auto owning() const {
        return [*this] { return m_gain; };
    }

private:
    float m_gain{0.5f};
};

```



```

class Fader {
public:
    // F.54: in a member function a `[=]` default does NOT copy the
    // members. It copies the invisible `this` pointer, so every member
    // access is through a reference to an object you may not own any
    // more. C++20 deprecated the silent version -- say what you mean:
    auto explicit_this() const {
        return [=, this] { return m_gain; };
    }

    // ...or take the whole object with you.
    auto owning() const {
        return [*this] { return m_gain; };
    }

private:
    float m_gain{0.5f};
};

```

```
// MUST FAIL: F.54. In a member function, `[=]` does not copy the  
// members -- it copies `this`. C++20 deprecated saying it silently,  
// so with warnings-as-errors this no longer builds.
```

```
class Fader {  
public:  
    auto gain_getter() const {  
        return [=] { return m_gain; };  
    }  
  
private:  
    float m_gain{0.5f};  
};  
  
int main() {  
    return Fader{}.gain_getter() > 0.f ? 0 : 1;  
}
```

```
// MUST FAIL: F.54. In a member function, `[=]` does not copy the
// members -- it copies `this`. C++20 deprecated saying it silently,
// so with warnings-as-errors this no longer builds.
```

```
class Fader {
public:
    auto gain_getter() const {
        return [=] { return m_gain; };
    }

private:
    float m_gain{0.5f};
};

int main() {
    return Fader{}.gain_getter() > 0.f ? 0 : 1;
}
```


C++20: capturing a pack

```
// C++20: capture a parameter pack by init-capture.  
template <class... Ts> auto defer_sum(Ts... ts) {  
    return [... args = std::move(ts)] { return (args + ... + 0); };  
}
```

C++20: capturing a pack

```
// C++20: capture a parameter pack by init-capture.  
template <class... Ts> auto defer_sum(Ts... ts) {  
    return [... args = std::move(ts)] { return (args + ... + 0); };  
}
```

F.52 and F.53: the rule that matters

- **F.52** Prefer capturing by reference in lambdas that will be used locally, including passed to algorithms
- **F.53** Avoid capturing by reference in lambdas that will be used non-locally, including returned, stored on the heap, or passed to another thread

Generic lambdas

C++14, C++20, C++23

C++14: `auto` in the parameter list

```
// C++14. `auto` in the parameter list makes operator() a template,  
// so this one closure works for every type you call it with.  
auto print_size = [](auto x) { return sizeof(x); };  
  
static_assert(print_size(1) == sizeof(int));  
static_assert(print_size(1.0) == sizeof(double));
```


C++14: `auto` in the parameter list

```
// C++14. `auto` in the parameter list makes operator() a template,  
// so this one closure works for every type you call it with.  
auto print_size = [](auto x) { return sizeof(x); };
```

```
static_assert(print_size(1) == sizeof(int));  
static_assert(print_size(1.0) == sizeof(double));
```

```
// C++20 gives the template parameter a name, which you need as soon
// as you want to *say* something about the type.
auto decay_copy = []<class T>(T&& x) { return std::decay_t<T>{std::forward<T>(x)}; };

// It also lets you constrain, and to take a pack.
auto sum = []<class... Ts>(Ts... xs)
    requires(std::is_arithmetic_v<Ts> && ...)
{ return (xs + ... + 0); };
```

```
// C++20 gives the template parameter a name, which you need as soon
// as you want to *say* something about the type.
auto decay_copy = []<class T>(T&& x) { return std::decay_t<T>{std::forward<T>(x)}; };

// It also lets you constrain, and to take a pack.
auto sum = []<class... Ts>(Ts... xs)
    requires(std::is_arithmetic_v<Ts> && ...)
{ return (xs + ... + 0); };

```



```
// C++20 gives the template parameter a name, which you need as soon
// as you want to *say* something about the type.
auto decay_copy = []<class T>(T&& x) { return std::decay_t<T>{std::forward<T>(x)}; };

// It also lets you constrain, and to take a pack.
auto sum = []<class... Ts>(Ts... xs)
    requires(std::is_arithmetic_v<Ts> && ...)
{ return (xs + ... + 0); };
```

Explicit template arguments

```
// A generic lambda is callable with explicit template arguments --  
// you just have to name operator() to do it.  
auto make = []<class T>() { return T{}; };  
  
auto zero = make.operator()<int>();
```

Explicit template arguments

```
// A generic lambda is callable with explicit template arguments --  
// you just have to name operator() to do it.  
auto make = []<class T>() { return T{}; };  
  
auto zero = make.operator<int>();
```

Explicit template arguments

```
// A generic lambda is callable with explicit template arguments --  
// you just have to name operator() to do it.  
auto make = []<class T>() { return T{}; };  
  
auto zero = make.operator<int>();
```

C.170: you cannot overload a lambda

```
// C.170: you cannot overload a lambda, because the name is a variable  
// and there is only one of it. A generic lambda plus if constexpr is  
// the sanctioned workaround.
```

```
auto describe = [<class T>(const T& x) → std::string {  
    if constexpr (std::is_integral_v<T>) {  
        return "integer " + std::to_string(x);  
    } else if constexpr (std::is_floating_point_v<T>) {  
        return "real " + std::to_string(x);  
    } else {  
        return std::string{x};  
    }  
};
```


C.170: you cannot overload a lambda

```
// C.170: you cannot overload a lambda, because the name is a variable  
// and there is only one of it. A generic lambda plus if constexpr is  
// the sanctioned workaround.
```

```
auto describe = [<class T>(const T& x) → std::string {  
    if constexpr (std::is_integral_v<T>) {  
        return "integer " + std::to_string(x);  
    } else if constexpr (std::is_floating_point_v<T>) {  
        return "real " + std::to_string(x);  
    } else {  
        return std::string{x};  
    }  
};
```

C.170: you cannot overload a lambda

```
// C.170: you cannot overload a lambda, because the name is a variable  
// and there is only one of it. A generic lambda plus if constexpr is  
// the sanctioned workaround.
```

```
auto describe = [<class T>(const T& x) → std::string {  
    if constexpr (std::is_integral_v<T>) {  
        return "integer " + std::to_string(x);  
    } else if constexpr (std::is_floating_point_v<T>) {  
        return "real " + std::to_string(x);  
    } else {  
        return std::string{x};  
    }  
};
```

C.170: you cannot overload a lambda

```
// C.170: you cannot overload a lambda, because the name is a variable  
// and there is only one of it. A generic lambda plus if constexpr is  
// the sanctioned workaround.
```

```
auto describe = []<class T>(const T& x) → std::string {  
    if constexpr (std::is_integral_v<T>) {  
        return "integer " + std::to_string(x);  
    } else if constexpr (std::is_floating_point_v<T>) {  
        return "real " + std::to_string(x);  
    } else {  
        return std::string{x};  
    }  
};
```


C.170: you cannot overload a lambda

```
// C.170: you cannot overload a lambda, because the name is a variable  
// and there is only one of it. A generic lambda plus if constexpr is  
// the sanctioned workaround.
```

```
auto describe = [<class T>(const T& x) → std::string {  
    if constexpr (std::is_integral_v<T>) {  
        return "integer " + std::to_string(x);  
    } else if constexpr (std::is_floating_point_v<T>) {  
        return "real " + std::to_string(x);  
    } else {  
        return std::string{x};  
    }  
};
```

The overload pattern

```
template <class... Ts> struct overloaded : Ts... {  
    using Ts::operator()...;  
};
```

// C++17 needs this deduction guide; C++20 onward it is implicit.

```
template <class... Ts> overloaded(Ts...) → overloaded<Ts...>;
```

The overload pattern

```
template <class... Ts> struct overloaded : Ts... {  
    using Ts::operator() ...;  
};
```

```
// C++17 needs this deduction guide; C++20 onward it is implicit.  
template <class... Ts> overloaded(Ts...) → overloaded<Ts...>;
```

The overload pattern

```
template <class... Ts> struct overloaded : Ts... {  
    using Ts::operator()...;  
};
```

```
// C++17 needs this deduction guide; C++20 onward it is implicit.  
template <class... Ts> overloaded(Ts...) → overloaded<Ts...>;
```

The overload pattern

```
template <class... Ts> struct overloaded : Ts... {  
    using Ts::operator()...;  
};
```

// C++17 needs this deduction guide; C++20 onward it is implicit.

```
template <class... Ts> overloaded(Ts...) → overloaded<Ts...>;
```


...gives you a real overload set to visit with

```
using value = std::variant<int, std::string>;
```

```
std::string describe(const value& v) {  
    return std::visit(overloaded{  
        [](int i) { return "int:" + std::to_string(i); },  
        [](const std::string& s) { return "string:" + s; },  
    },  
    v);  
}
```

...gives you a real overload set to visit with

```
using value = std::variant<int, std::string>;

std::string describe(const value& v) {
    return std::visit(overloaded{
        [](int i) { return "int:" + std::to_string(i); },
        [](const std::string& s) { return "string:" + s; },
    },
    v);
}
```

...gives you a real overload set to visit with

```
using value = std::variant<int, std::string>;

std::string describe(const value& v) {
    return std::visit(overloaded{
        [](int i) { return "int:" + std::to_string(i); },
        [](const std::string& s) { return "string:" + s; },
    },
    v);
}
```


...gives you a real overload set to visit with

```
using value = std::variant<int, std::string>;

std::string describe(const value& v) {
    return std::visit(overloaded{
        [](int i) { return "int:" + std::to_string(i); },
        [](const std::string& s) { return "string:" + s; },
    },
    v);
}
```

The `if constexpr` visitor

```
std::string print_my_things(const MyThings& thing) {  
    return std::visit(  
        [](auto x) → std::string {  
            if constexpr (std::same_as<decltype(x), std::optional<int>>) {  
                return x ? std::to_string(*x) : "empty";  
            } else if constexpr (std::same_as<decltype(x), std::string>) {  
                return x;  
            } else {  
                return std::to_string(x);  
            }  
        },  
        thing);  
}
```

The `if constexpr` visitor

```
std::string print_my_things(const MyThings& thing) {  
    return std::visit(  
        [](auto x) → std::string {  
            if constexpr (std::same_as<decltype(x), std::optional<int>>) {  
                return x ? std::to_string(*x) : "empty";  
            } else if constexpr (std::same_as<decltype(x), std::string>) {  
                return x;  
            } else {  
                return std::to_string(x);  
            }  
        },  
        thing);  
}
```

The `if constexpr` visitor

```
std::string print_my_things(const MyThings& thing) {  
    return std::visit(  
        [](auto x) → std::string {  
            if constexpr (std::same_as<decltype(x), std::optional<int>>) {  
                return x ? std::to_string(*x) : "empty";  
            } else if constexpr (std::same_as<decltype(x), std::string>) {  
                return x;  
            } else {  
                return std::to_string(x);  
            }  
        },  
        thing);  
}
```

The `if constexpr` visitor

```
std::string print_my_things(const MyThings& thing) {  
    return std::visit(  
        [](auto x) → std::string {  
            if constexpr (std::same_as<decltype(x), std::optional<int>>) {  
                return x ? std::to_string(*x) : "empty";  
            } else if constexpr (std::same_as<decltype(x), std::string>) {  
                return x;  
            } else {  
                return std::to_string(x);  
            }  
        },  
        thing);  
}
```


The `if constexpr` visitor

```
std::string print_my_things(const MyThings& thing) {  
    return std::visit(  
        [](auto x) → std::string {  
            if constexpr (std::same_as<decltype(x), std::optional<int>>) {  
                return x ? std::to_string(*x) : "empty";  
            } else if constexpr (std::same_as<decltype(x), std::string>) {  
                return x;  
            } else {  
                return std::to_string(x);  
            }  
        },  
        thing);  
}
```

```
// C++20: naming the type is worth it here -- `decltype(x)` on an
// `auto` parameter drags the value category around with it, and one
// day you will write `const auto&` and wonder why nothing matches.
std::string print_my_things_20(const MyThings& thing) {
    return std::visit(
        [<class T>(const T& x) → std::string {
            if constexpr (std::same_as<T, std::optional<int>>) {
                return x ? std::to_string(*x) : "empty";
            } else if constexpr (std::same_as<T, std::string>) {
                return x;
            } else {
                return std::to_string(x);
            }
        },
        thing);
}
```

```

// C++20: naming the type is worth it here -- `decltype(x)` on an
// `auto` parameter drags the value category around with it, and one
// day you will write `const auto&` and wonder why nothing matches.
std::string print_my_things_20(const MyThings& thing) {
    return std::visit(
        []<class T>(const T& x) → std::string {
            if constexpr (std::same_as<T, std::optional<int>>) {
                return x ? std::to_string(*x) : "empty";
            } else if constexpr (std::same_as<T, std::string>) {
                return x;
            } else {
                return std::to_string(x);
            }
        },
        thing);
}

```



```

// C++20: naming the type is worth it here -- `decltype(x)` on an
// `auto` parameter drags the value category around with it, and one
// day you will write `const auto&` and wonder why nothing matches.
std::string print_my_things_20(const MyThings& thing) {
    return std::visit(
        [<class T>(const T& x) → std::string {
            if constexpr (std::same_as<T, std::optional<int>>) {
                return x ? std::to_string(*x) : "empty";
            } else if constexpr (std::same_as<T, std::string>) {
                return x;
            } else {
                return std::to_string(x);
            }
        },
        thing);
}

```

```

// C++20: naming the type is worth it here -- `decltype(x)` on an
// `auto` parameter drags the value category around with it, and one
// day you will write `const auto&` and wonder why nothing matches.
std::string print_my_things_20(const MyThings& thing) {
    return std::visit(
        []<class T>(const T& x) → std::string {
            if constexpr (std::same_as<T, std::optional<int>>) {
                return x ? std::to_string(*x) : "empty";
            } else if constexpr (std::same_as<T, std::string>) {
                return x;
            } else {
                return std::to_string(x);
            }
        },
        thing);
}

```

A lambda cannot name itself

```
// The closure cannot refer to itself, so we gave it a name that is
// not the closure: a std::function, captured by reference. It works,
// it type-erases every call, and it dangles the moment `factorial`
// leaves scope.
int slow_factorial(int n) {
    std::function<int(int)> factorial = [&factorial](int k) → int {
        return k ≤ 1 ? 1 : k * factorial(k - 1);
    };

    return factorial(n);
}
```

A lambda cannot name itself

```
// The closure cannot refer to itself, so we gave it a name that is
// not the closure: a std::function, captured by reference. It works,
// it type-erases every call, and it dangles the moment `factorial`
// leaves scope.
int slow_factorial(int n) {
    std::function<int(int)> factorial = [&factorial](int k) → int {
        return k ≤ 1 ? 1 : k * factorial(k - 1);
    };

    return factorial(n);
}
```

A lambda cannot name itself

```
// The closure cannot refer to itself, so we gave it a name that is
// not the closure: a std::function, captured by reference. It works,
// it type-erases every call, and it dangles the moment `factorial`
// leaves scope.
int slow_factorial(int n) {
    std::function<int(int)> factorial = [&factorial](int k) → int {
        return k ≤ 1 ? 1 : k * factorial(k - 1);
    };

    return factorial(n);
}
```


A lambda cannot name itself

```
// The closure cannot refer to itself, so we gave it a name that is
// not the closure: a std::function, captured by reference. It works,
// it type-erases every call, and it dangles the moment `factorial`
// leaves scope.
int slow_factorial(int n) {
    std::function<int(int)> factorial = [&factorial](int k) → int {
        return k ≤ 1 ? 1 : k * factorial(k - 1);
    };

    return factorial(n);
}
```

C++23: the lambda is handed itself

```
// C++23: the lambda is handed itself. No type erasure, no allocation,  
// nothing to dangle -- and it can be constexpr.  
constexpr auto factorial = [](this auto&& self, int n) → int {  
    return n ≤ 1 ? 1 : n * self(n - 1);  
};  
  
static_assert(factorial(5) == 120);
```

C++23: the lambda is handed itself

```
// C++23: the lambda is handed itself. No type erasure, no allocation,  
// nothing to dangle -- and it can be constexpr.  
constexpr auto factorial = [](this auto&& self, int n) → int {  
    return n ≤ 1 ? 1 : n * self(n - 1);  
};  
  
static_assert(factorial(5) == 120);
```


C++23: the lambda is handed itself

```
// C++23: the lambda is handed itself. No type erasure, no allocation,  
// nothing to dangle -- and it can be constexpr.  
constexpr auto factorial = [](this auto&& self, int n) → int {  
    return n ≤ 1 ? 1 : n * self(n - 1);  
};  
  
static_assert(factorial(5) == 120);
```

C++23: the lambda is handed itself

```
// C++23: the lambda is handed itself. No type erasure, no allocation,  
// nothing to dangle -- and it can be constexpr.  
constexpr auto factorial = [](this auto&& self, int n) → int {  
    return n ≤ 1 ? 1 : n * self(n - 1);  
};
```

```
static_assert(factorial(5) == 120);
```

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {
    auto engine = std::mt19937{seed};
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};
    const auto gain = 0.1f;

    auto buffer = std::vector<float>(frames);

    // F.52: capture by reference when the lambda is used right here.
    // The engine has to advance, so a copy would be wrong anyway.
    std::generate(buffer.begin(), buffer.end(), [&] { return gain * dist(engine); });

    return buffer;
}
```

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto engine = std::mt19937{seed};  
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};  
    const auto gain = 0.1f;  
  
    auto buffer = std::vector<float>(frames);  
  
    // F.52: capture by reference when the lambda is used right here.  
    // The engine has to advance, so a copy would be wrong anyway.  
    std::generate(buffer.begin(), buffer.end(), [&] { return gain * dist(engine); });  
  
    return buffer;  
}
```

```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto engine = std::mt19937{seed};  
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};  
    const auto gain = 0.1f;  
  
    auto buffer = std::vector<float>(frames);  
  
    // F.52: capture by reference when the lambda is used right here.  
    // The engine has to advance, so a copy would be wrong anyway.  
    std::generate(buffer.begin(), buffer.end(), [&] { return gain * dist(engine); });  
  
    return buffer;  
}
```



```
std::vector<float> make_noise(std::size_t frames, unsigned seed) {  
    auto engine = std::mt19937{seed};  
    auto dist = std::uniform_real_distribution<float>{-1.f, 1.f};  
    const auto gain = 0.1f;  
  
    auto buffer = std::vector<float>(frames);  
  
    // F.52: capture by reference when the lambda is used right here.  
    // The engine has to advance, so a copy would be wrong anyway.  
    std::generate(buffer.begin(), buffer.end(), [&] { return gain * dist(engine); });  
  
    return buffer;  
}
```

```
// In a processor the state outlives the call, so it goes back to
// being a member -- and the lambda captures `this`.
class NoiseProcessor {
public:
    explicit NoiseProcessor(unsigned seed) : m_gen{seed} {}

    void ProcessInPlace(std::vector<float>& buffer) {
        std::generate(buffer.begin(), buffer.end(), [this] { return m_gain * m_dist(m_gen); });
    }

    void SetGain(float gain) { m_gain = gain; }

private:
    std::mt19937 m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
    float m_gain{0.1f};
};
```

```

// In a processor the state outlives the call, so it goes back to
// being a member -- and the lambda captures `this`.
class NoiseProcessor {
public:
    explicit NoiseProcessor(unsigned seed) : m_gen{seed} {}

    void ProcessInPlace(std::vector<float>& buffer) {
        std::generate(buffer.begin(), buffer.end(), [this] { return m_gain * m_dist(m_gen); });
    }

    void SetGain(float gain) { m_gain = gain; }

private:
    std::mt19937 m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
    float m_gain{0.1f};
};

```



```

// In a processor the state outlives the call, so it goes back to
// being a member -- and the lambda captures `this`.
class NoiseProcessor {
public:
    explicit NoiseProcessor(unsigned seed) : m_gen{seed} {}

    void ProcessInPlace(std::vector<float>& buffer) {
        std::generate(buffer.begin(), buffer.end(), [this] { return m_gain * m_dist(m_gen); });
    }

    void SetGain(float gain) { m_gain = gain; }

private:
    std::mt19937 m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
    float m_gain{0.1f};
};

```

```

// In a processor the state outlives the call, so it goes back to
// being a member -- and the lambda captures `this`.
class NoiseProcessor {
public:
    explicit NoiseProcessor(unsigned seed) : m_gen{seed} {}

    void ProcessInPlace(std::vector<float>& buffer) {
        std::generate(buffer.begin(), buffer.end(), [this] { return m_gain * m_dist(m_gen); });
    }

    void SetGain(float gain) { m_gain = gain; }

private:
    std::mt19937 m_gen;
    std::uniform_real_distribution<float> m_dist{-1.f, 1.f};
    float m_gain{0.1f};
};

```

Capture-less lambdas get extra powers

```
// A capture-less lambda converts to a plain function pointer...
auto is_even = [](int n) { return n % 2 == 0; };
static_assert(std::is_convertible_v<decltype(is_even), bool (*)(int)>);

// ...and since C++20 it is default-constructible, so it can be a
// comparator.
```

Capture-less lambdas get extra powers

```
// A capture-less lambda converts to a plain function pointer...
auto is_even = [](int n) { return n % 2 == 0; };
static_assert(std::is_convertible_v<decltype(is_even), bool (*)(int)>);

// ...and since C++20 it is default-constructible, so it can be a
// comparator.
```

Capture-less lambdas get extra powers

```
// A capture-less lambda converts to a plain function pointer...
auto is_even = [](int n) { return n % 2 == 0; };
static_assert(std::is_convertible_v<decltype(is_even), bool (*)(int)>);

// ...and since C++20 it is default-constructible, so it can be a
// comparator.
```


Capture-less lambdas get extra powers

```
// A capture-less lambda converts to a plain function pointer...
auto is_even = [](int n) { return n % 2 == 0; };
static_assert(std::is_convertible_v<decltype(is_even), bool (*)(int)>);

// ...and since C++20 it is default-constructible, so it can be a
// comparator.
```

...which is how this works

```
auto by_abs = [](int a, int b) { return (a < 0 ? -a : a) < (b < 0 ? -b : b); };
```

```
// CTAD deduces std::set<int, decltype(by_abs)>.
std::set numbers{{-3, 1, -2}, by_abs};
```

...which is how this works

```
auto by_abs = [](int a, int b) { return (a < 0 ? -a : a) < (b < 0 ? -b : b); };
```

```
// CTAD deduces std::set<int, decltype(by_abs)>.
```

```
std::set numbers{{-3, 1, -2}, by_abs};
```


...which is how this works

```
auto by_abs = [](int a, int b) { return (a < 0 ? -a : a) < (b < 0 ? -b : b); };
```

```
// CTAD deduces std::set<int, decltype(by_abs)>.
```

```
std::set numbers{{-3, 1, -2}, by_abs};
```

std::function

...and what it costs

So what *is* the type of a lambda?

```
// Every lambda expression has its own closure type. These two are
// character-for-character identical and still different types.
auto first = [](int n) { return n * 2; };
auto second = [](int n) { return n * 2; };

static_assert(!std::is_same_v<decltype(first), decltype(second)>);

// The type has no name you can spell, so `auto` is not a shortcut
// here -- it is the only option.
using closure_type = decltype(first);
```

So what *is* the type of a lambda?

```
// Every lambda expression has its own closure type. These two are
// character-for-character identical and still different types.
auto first = [](int n) { return n * 2; };
auto second = [](int n) { return n * 2; };

static_assert(!std::is_same_v<decltype(first), decltype(second)>);

// The type has no name you can spell, so `auto` is not a shortcut
// here -- it is the only option.
using closure_type = decltype(first);
```

So what **is** the type of a lambda?

```
// Every lambda *expression* has its own closure type. These two are  
// character-for-character identical and still different types.
```

```
auto first = [](int n) { return n * 2; };  
auto second = [](int n) { return n * 2; };
```

```
static_assert(!std::is_same_v<decltype(first), decltype(second)>);
```

```
// The type has no name you can spell, so `auto` is not a shortcut  
// here -- it is the only option.
```

```
using closure_type = decltype(first);
```


So what **is** the type of a lambda?

```
// Every lambda *expression* has its own closure type. These two are
// character-for-character identical and still different types.
auto first = [](int n) { return n * 2; };
auto second = [](int n) { return n * 2; };

static_assert(!std::is_same_v<decltype(first), decltype(second)>);

// The type has no name you can spell, so `auto` is not a shortcut
// here -- it is the only option.
using closure_type = decltype(first);
```

...which is a problem the moment you want two of them

```
// Which is a problem the moment you want more than one of them.  
std::vector<closure_type> only_clones; // a vector of *that* lambda  
  
// You cannot write the type these two have in common, because they  
// do not have one. That is what type erasure is for.
```

...which is a problem the moment you want two of them

```
// Which is a problem the moment you want more than one of them.
```

```
std::vector<closure_type> only_clones; // a vector of *that* lambda
```

```
// You cannot write the type these two have in common, because they  
// do not have one. That is what type erasure is for.
```


...which is a problem the moment you want two of them

```
// Which is a problem the moment you want more than one of them.  
std::vector<closure_type> only_clones; // a vector of *that* lambda  
  
// You cannot write the type these two have in common, because they  
// do not have one. That is what type erasure is for.
```

What you actually want to say

```
// What you usually want to say about a callable is not its type but  
// what you can do with it.  
static_assert(std::is_invocable_v<closure_type, int>);  
static_assert(std::is_invocable_r_v<int, closure_type, int>);
```

What you actually want to say

```
// What you usually want to say about a callable is not its type but  
// what you can do with it.  
static_assert(std::is_invocable_v<closure_type, int>);  
static_assert(std::is_invocable_r_v<int, closure_type, int>);
```

What you actually want to say

```
// What you usually want to say about a callable is not its type but  
// what you can do with it.  
static_assert(std::is_invocable_v<closure_type, int>);  
static_assert(std::is_invocable_r_v<int, closure_type, int>);
```

```
void anything() {  
    // A function pointer...  
    std::function<int(int)> a = twice;  
  
    // ...a function object...  
    std::function<int(int)> b = Thrice{};  
  
    // ...a lambda, capture and all...  
    int factor = 4;  
    std::function<int(int)> c = [factor](int n) { return n * factor; };  
  
    // ...and a member function, given something to call it on.  
    std::function<std::size_t(const std::string&)> d = &std::string::size;  
  
    assert(a(21) == 42);  
    assert(b(14) == 42);  
    assert(c(10) == 40);  
    assert(d("hello") == 5u);  
}
```



```
void anything() {  
    // A function pointer...  
    std::function<int(int)> a = twice;  
  
    // ...a function object...  
    std::function<int(int)> b = Thrice{};  
  
    // ...a lambda, capture and all...  
    int factor = 4;  
    std::function<int(int)> c = [factor](int n) { return n * factor; };  
  
    // ...and a member function, given something to call it on.  
    std::function<std::size_t(const std::string&)> d = &std::string::size;  
  
    assert(a(21) == 42);  
    assert(b(14) == 42);  
    assert(c(10) == 40);  
    assert(d("hello") == 5u);  
}
```

```
void anything() {  
    // A function pointer...  
    std::function<int(int)> a = twice;  
  
    // ...a function object...  
    std::function<int(int)> b = Thrice{};  
  
    // ...a lambda, capture and all...  
    int factor = 4;  
    std::function<int(int)> c = [factor](int n) { return n * factor; };  
  
    // ...and a member function, given something to call it on.  
    std::function<std::size_t(const std::string&)> d = &std::string::size;  
  
    assert(a(21) == 42);  
    assert(b(14) == 42);  
    assert(c(10) == 40);  
    assert(d("hello") == 5u);  
}
```

```
void anything() {  
    // A function pointer...  
    std::function<int(int)> a = twice;  
  
    // ...a function object...  
    std::function<int(int)> b = Thrice{};  
  
    // ...a lambda, capture and all...  
    int factor = 4;  
    std::function<int(int)> c = [factor](int n) { return n * factor; };  
  
    // ...and a member function, given something to call it on.  
    std::function<std::size_t(const std::string&)> d = &std::string::size;  
  
    assert(a(21) == 42);  
    assert(b(14) == 42);  
    assert(c(10) == 40);  
    assert(d("hello") == 5u);  
}
```



```
void anything() {
    // A function pointer...
    std::function<int(int)> a = twice;

    // ...a function object...
    std::function<int(int)> b = Thrice{};

    // ...a lambda, capture and all...
    int factor = 4;
    std::function<int(int)> c = [factor](int n) { return n * factor; };

    // ...and a member function, given something to call it on.
    std::function<std::size_t(const std::string&)> d = &std::string::size;

    assert(a(21) == 42);
    assert(b(14) == 42);
    assert(c(10) == 40);
    assert(d("hello") == 5u);
}
```

This is the thing closures could not do

```
// This is the thing the closure types could not do: sit in a  
// container together.
```

```
std::vector<std::function<int(int)>> pipeline() {  
    return {twice, Thrice{}, [](int n) { return n + 1; }};  
}
```

This is the thing closures could not do

```
// This is the thing the closure types could not do: sit in a  
// container together.
```

```
std::vector<std::function<int(int)>> pipeline() {  
    return {twice, Thrice{}, [](int n) { return n + 1; }};  
}
```

This is the thing closures could not do

```
// This is the thing the closure types could not do: sit in a  
// container together.
```

```
std::vector<std::function<int(int)>> pipeline() {  
    return {twice, Thrice{}, [](int n) { return n + 1; }};  
}
```

```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```



```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```

```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```

```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```



```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```

```

void emptiness() {
    std::function<int(int)> f; // default-constructed: no target

    assert(!f);
    assert(f == nullptr);

    // Calling an empty one is not undefined -- it throws.
    try {
        f(1);
        assert(false);
    } catch (const std::bad_function_call&) {
    }

    // A *null function pointer* is recognised and leaves it empty.
    int (*null_fp)(int) = nullptr;
    f = null_fp;
    assert(!f);

    f = twice;
    assert(f);

    f = nullptr; // and this empties it again
    assert(!f);
}

```

You can ask what is inside

```
void introspection() {  
    std::function<int(int)> f = twice;  
  
    // You can ask what is in there, if you already know what to ask.  
    assert(f.target_type() == typeid(int (*)(int)));  
    assert(f.target<int (*)(int)>() != nullptr);  
    assert(f.target<Thrice>() == nullptr);  
}
```

You can ask what is inside

```
void introspection() {  
    std::function<int(int)> f = twice;  
  
    // You can ask what is in there, if you already know what to ask.  
    assert(f.target_type() == typeid(int (*)(int)));  
    assert(f.target<int (*)(int)>() != nullptr);  
    assert(f.target<Thrice>() == nullptr);  
}
```

You can ask what is inside

```
void introspection() {  
    std::function<int(int)> f = twice;  
  
    // You can ask what is in there, if you already know what to ask.  
    assert(f.target_type() == typeid(int (*)(int)));  
    assert(f.target<int (*)(int)>() != nullptr);  
    assert(f.target<Thrice>() == nullptr);  
}
```


You can ask what is inside

```
void introspection() {  
    std::function<int(int)> f = twice;  
  
    // You can ask what is in there, if you already know what to ask.  
    assert(f.target_type() == typeid(int (*)(int)));  
    assert(f.target<int (*)(int)>() != nullptr);  
    assert(f.target<Thrice>() == nullptr);  
}
```

How does that work?

Let's write it

The part that does not know the type

```
// The part that does not depend on the callable's type.  
template <class R, class... Args> struct callable_base {  
    virtual ~callable_base() = default;  
    virtual R invoke(Args...) const = 0;  
    virtual std::unique_ptr<callable_base> clone() const = 0;  
};
```


The part that does not know the type

```
// The part that does not depend on the callable's type.  
template <class R, class... Args> struct callable_base {  
    virtual ~callable_base() = default;  
    virtual R invoke(Args...) const = 0;  
    virtual std::unique_ptr<callable_base> clone() const = 0;  
};
```

The part that does not know the type

```
// The part that does not depend on the callable's type.  
template <class R, class... Args> struct callable_base {  
    virtual ~callable_base() = default;  
    virtual R invoke(Args...) const = 0;  
    virtual std::unique_ptr<callable_base> clone() const = 0;  
};
```

The part that does not know the type

```
// The part that does not depend on the callable's type.  
template <class R, class... Args> struct callable_base {  
    virtual ~callable_base() = default;  
    virtual R invoke(Args...) const = 0;  
    virtual std::unique_ptr<callable_base> clone() const = 0;  
};
```

The part that does

```
// One of these is instantiated per callable you store. This is where
// the type goes -- and where it stops.
template <class F, class R, class... Args>
struct callable_holder final : callable_base<R, Args...> {
    explicit callable_holder(F f) : m_f{std::move(f)} {}

    R invoke(Args... args) const override { return m_f(args...); }

    std::unique_ptr<callable_base<R, Args...>> clone() const override {
        return std::make_unique<callable_holder>(m_f);
    }
}

// Note the `mutable`: invoke() is const, but a mutable lambda's
// operator() is not. std::function has exactly this hole.
```

The part that does

```
// One of these is instantiated per callable you store. This is where
// the type goes -- and where it stops.
template <class F, class R, class... Args>
struct callable_holder final : callable_base<R, Args...> {
    explicit callable_holder(F f) : m_f{std::move(f)} {}

    R invoke(Args... args) const override { return m_f(args...); }

    std::unique_ptr<callable_base<R, Args...>> clone() const override {
        return std::make_unique<callable_holder>(m_f);
    }
}

// Note the `mutable`: invoke() is const, but a mutable lambda's
// operator() is not. std::function has exactly this hole.
```


The part that does

```
// One of these is instantiated per callable you store. This is where
// the type goes -- and where it stops.
template <class F, class R, class... Args>
struct callable_holder final : callable_base<R, Args...> {
    explicit callable_holder(F f) : m_f{std::move(f)} {}

    R invoke(Args... args) const override { return m_f(args...); }

    std::unique_ptr<callable_base<R, Args...>> clone() const override {
        return std::make_unique<callable_holder>(m_f);
    }
}

// Note the `mutable`: invoke() is const, but a mutable lambda's
// operator() is not. std::function has exactly this hole.
```

The part that does

```
// One of these is instantiated per callable you store. This is where
// the type goes -- and where it stops.
template <class F, class R, class... Args>
struct callable_holder final : callable_base<R, Args...> {
    explicit callable_holder(F f) : m_f{std::move(f)} {}

    R invoke(Args... args) const override { return m_f(args...); }

    std::unique_ptr<callable_base<R, Args...>> clone() const override {
        return std::make_unique<callable_holder>(m_f);
    }
}

// Note the `mutable`: invoke() is const, but a mutable lambda's
// operator() is not. std::function has exactly this hole.
```

The part that does

```
// One of these is instantiated per callable you store. This is where
// the type goes -- and where it stops.
template <class F, class R, class... Args>
struct callable_holder final : callable_base<R, Args...> {
    explicit callable_holder(F f) : m_f{std::move(f)} {}

    R invoke(Args... args) const override { return m_f(args...); }

    std::unique_ptr<callable_base<R, Args...>> clone() const override {
        return std::make_unique<callable_holder>(m_f);
    }
}

// Note the `mutable`: invoke() is const, but a mutable lambda's
// operator() is not. std::function has exactly this hole.
```



```

template <class> class my_function;

template <class R, class... Args> class my_function<R(Args...)> {
public:
    my_function() = default;

    template <class F>
    my_function(F f) : m_target{std::make_unique<callable_holder<F, R, Args...>>(std::move(f))} {}

    my_function(const my_function& other) : m_target{other ? other.m_target→clone() : nullptr} {}
    my_function(my_function&&) = default;

    explicit operator bool() const { return m_target ≠ nullptr; }

    R operator()(Args... args) const {
        if (!m_target) {
            throw std::bad_function_call{};
        }
        return m_target→invoke(args...);
    }

private:
    std::unique_ptr<callable_base<R, Args...>> m_target;
};

```

```

template <class> class my_function;

template <class R, class... Args> class my_function<R(Args...)> {
public:
    my_function() = default;

    template <class F>
    my_function(F f) : m_target{std::make_unique<callable_holder<F, R, Args...>>(std::move(f))} {}

    my_function(const my_function& other) : m_target{other ? other.m_target→clone() : nullptr} {}
    my_function(my_function&&) = default;

    explicit operator bool() const { return m_target ≠ nullptr; }

    R operator()(Args... args) const {
        if (!m_target) {
            throw std::bad_function_call{};
        }
        return m_target→invoke(args...);
    }

private:
    std::unique_ptr<callable_base<R, Args...>> m_target;
};

```

```

template <class> class my_function;

template <class R, class... Args> class my_function<R(Args...)> {
public:
    my_function() = default;

    template <class F>
    my_function(F f) : m_target{std::make_unique<callable_holder<F, R, Args...>>(std::move(f))} {}

    my_function(const my_function& other) : m_target{other ? other.m_target→clone() : nullptr} {}
    my_function(my_function&&) = default;

    explicit operator bool() const { return m_target ≠ nullptr; }

    R operator()(Args... args) const {
        if (!m_target) {
            throw std::bad_function_call{};
        }
        return m_target→invoke(args...);
    }

private:
    std::unique_ptr<callable_base<R, Args...>> m_target;
};

```

```

template <class> class my_function;

template <class R, class... Args> class my_function<R(Args...)> {
public:
    my_function() = default;

    template <class F>
    my_function(F f) : m_target{std::make_unique<callable_holder<F, R, Args...>>(std::move(f))} {}

    my_function(const my_function& other) : m_target{other ? other.m_target→clone() : nullptr} {}
    my_function(my_function&&) = default;

    explicit operator bool() const { return m_target ≠ nullptr; }

    R operator()(Args... args) const {
        if (!m_target) {
            throw std::bad_function_call{};
        }
        return m_target→invoke(args...);
    }

private:
    std::unique_ptr<callable_base<R, Args...>> m_target;
};

```

```

template <class> class my_function;

template <class R, class... Args> class my_function<R(Args...)> {
public:
    my_function() = default;

    template <class F>
    my_function(F f) : m_target{std::make_unique<callable_holder<F, R, Args...>>(std::move(f))} {}

    my_function(const my_function& other) : m_target{other ? other.m_target→clone() : nullptr} {}
    my_function(my_function&&) = default;

    explicit operator bool() const { return m_target ≠ nullptr; }

    R operator()(Args... args) const {
        if (!m_target) {
            throw std::bad_function_call{};
        }
        return m_target→invoke(args...);
    }

private:
    std::unique_ptr<callable_base<R, Args...>> m_target;
};

```


How much does it actually cost? Count.

```
std::size_t allocations = 0;

void* operator new(std::size_t bytes) {
    ++allocations;
    return std::malloc(bytes);
}

void operator delete(void* p) noexcept {
    std::free(p);
}

void operator delete(void* p, std::size_t) noexcept {
    std::free(p);
}
```

How much does it actually cost? Count.

```
std::size_t allocations = 0;
```

```
void* operator new(std::size_t bytes) {  
    ++allocations;  
    return std::malloc(bytes);  
}
```

```
void operator delete(void* p) noexcept {  
    std::free(p);  
}
```

```
void operator delete(void* p, std::size_t) noexcept {  
    std::free(p);  
}
```

How much does it actually cost? Count.

```
std::size_t allocations = 0;
```

```
void* operator new(std::size_t bytes) {  
    ++allocations;  
    return std::malloc(bytes);  
}
```

```
void operator delete(void* p) noexcept {  
    std::free(p);  
}
```

```
void operator delete(void* p, std::size_t) noexcept {  
    std::free(p);  
}
```


Two closures, one guarantee

```
// The standard says nothing about a small-buffer optimisation.  
// Every implementation has one; none of them promise you a size.  
const auto before_empty = allocations;  
std::function<void()> empty_closure = [] {};  
const auto empty_cost = allocations - before_empty;  
  
const auto before_big = allocations;  
auto big = BigState{};  
std::function<void()> big_closure = [big] { (void)big; };  
const auto big_cost = allocations - before_big;
```

Two closures, one guarantee

```
// The standard says nothing about a small-buffer optimisation.  
// Every implementation has one; none of them promise you a size.  
const auto before_empty = allocations;  
std::function<void()> empty_closure = [] {};  
const auto empty_cost = allocations - before_empty;  
  
const auto before_big = allocations;  
auto big = BigState{};  
std::function<void()> big_closure = [big] { (void)big; };  
const auto big_cost = allocations - before_big;
```

Two closures, one guarantee

```
// The standard says nothing about a small-buffer optimisation.  
// Every implementation has one; none of them promise you a size.  
const auto before_empty = allocations;  
std::function<void()> empty_closure = [] {};  
const auto empty_cost = allocations - before_empty;  
  
const auto before_big = allocations;  
auto big = BigState{};  
std::function<void()> big_closure = [big] { (void)big; };  
const auto big_cost = allocations - before_big;
```

Two closures, one guarantee

```
// The standard says nothing about a small-buffer optimisation.  
// Every implementation has one; none of them promise you a size.  
const auto before_empty = allocations;  
std::function<void()> empty_closure = [] {};  
const auto empty_cost = allocations - before_empty;  
  
const auto before_big = allocations;  
auto big = BigState{};  
std::function<void()> big_closure = [big] { (void)big; };  
const auto big_cost = allocations - before_big;
```

So where is the buffer? Depends who you ask

- **libstdc++** — 16 bytes, and only if trivially copyable
 - a closure capturing a `std::string`, `shared_ptr` or `unique_ptr` always allocates, however small
- **libc++** — a 24-byte buffer, minus a `vptr`, so about 16 usable
 - and the target must be nothrow-copy-constructible
- **MSVC STL** — 56 bytes on x64, if the target is nothrow-move
- The same lambda allocates on one platform and not on another

And `const` does not mean what you think

```
// N4348: operator() is const, but the target only has to be callable  
// as a non-const lvalue -- so a const std::function calls the  
// non-const overload, and can mutate its target's state.
```

```
struct Both {  
    bool operator()() { return true; }  
    bool operator()() const { return false; }  
};
```

```
void the_const_hole() {  
    const std::function<bool()> f = Both{};  
    assert(f()); // the NON-const overload, through a const object  
  
    const std::function<int()> counter = [n = 0]() mutable { return ++n; };  
    assert(counter() == 1);
```

And `const` does not mean what you think

```
// N4348: operator() is const, but the target only has to be callable  
// as a non-const lvalue -- so a const std::function calls the  
// non-const overload, and can mutate its target's state.
```

```
struct Both {  
    bool operator()() { return true; }  
    bool operator()() const { return false; }  
};
```

```
void the_const_hole() {  
    const std::function<bool()> f = Both{};  
    assert(f()); // the NON-const overload, through a const object  
  
    const std::function<int()> counter = [n = 0]() mutable { return ++n; };  
    assert(counter() == 1);
```

And `const` does not mean what you think

```
// N4348: operator() is const, but the target only has to be callable  
// as a non-const lvalue -- so a const std::function calls the  
// non-const overload, and can mutate its target's state.
```

```
struct Both {  
    bool operator()() { return true; }  
    bool operator()() const { return false; }  
};
```

```
void the_const_hole() {  
    const std::function<bool()> f = Both{};  
    assert(f()); // the NON-const overload, through a const object  
  
    const std::function<int()> counter = [n = 0]() mutable { return ++n; };  
    assert(counter() == 1);
```


And `const` does not mean what you think

```
// N4348: operator() is const, but the target only has to be callable  
// as a non-const lvalue -- so a const std::function calls the  
// non-const overload, and can mutate its target's state.
```

```
struct Both {  
    bool operator()() { return true; }  
    bool operator()() const { return false; }  
};
```

```
void the_const_hole() {  
    const std::function<bool()> f = Both{};  
    assert(f()); // the NON-const overload, through a const object
```

```
const std::function<int()> counter = [n = 0]() mutable { return ++n; };  
assert(counter() == 1);
```

```
// MUST FAIL: std::function requires a copy-constructible target, so a
// closure that owns a unique_ptr cannot go in one. This is the hole
// std::move_only_function (C++23) fills.
```

```
#include <functional>
#include <memory>
```

```
int main() {
    auto owned = std::make_unique<int>(42);

    std::function<int()> f = [p = std::move(owned)] { return *p; };

    return f();
}
```

```
// MUST FAIL: std::function requires a copy-constructible target, so a
// closure that owns a unique_ptr cannot go in one. This is the hole
// std::move_only_function (C++23) fills.
```

```
#include <functional>
#include <memory>
```

```
int main() {
    auto owned = std::make_unique<int>(42);

    std::function<int()> f = [p = std::move(owned)] { return *p; };

    return f();
}
```

```
// MUST FAIL: std::function requires a copy-constructible target, so a  
// closure that owns a unique_ptr cannot go in one. This is the hole  
// std::move_only_function (C++23) fills.
```

```
#include <functional>  
#include <memory>
```

```
int main() {  
    auto owned = std::make_unique<int>(42);  
  
    std::function<int()> f = [p = std::move(owned)] { return *p; };  
  
    return f();  
}
```

C++23: `std::move\_only\_function`

```
// A closure that owns something can finally be stored.
std::move_only_function<int()> hold_a_unique_ptr() {
    auto owned = std::make_unique<int>(42);

    return [p = std::move(owned)] { return *p; };
}
```


C++23: `std::move\_only\_function`

```
// A closure that owns something can finally be stored.  
std::move_only_function<int()> hold_a_unique_ptr() {  
    auto owned = std::make_unique<int>(42);  
  
    return [p = std::move(owned)] { return *p; };  
}
```

C++23: `std::move\_only\_function`

```
// A closure that owns something can finally be stored.
std::move_only_function<int()> hold_a_unique_ptr() {
    auto owned = std::make_unique<int>(42);

    return [p = std::move(owned)] { return *p; };
}
```

```
// The cv- and ref-qualifiers live in the *signature*, so the wrapper
// can promise what the target actually supports.
void qualifiers() {
    // Callable on a const object, because the target is const-callable.
    std::move_only_function<int() const> readable = [] { return 42; };

    // Callable once, on an rvalue.
    std::move_only_function<int() &&> consumed = [] { return 42; };

    // And noexcept is part of it too.
    std::move_only_function<int() const noexcept> never_throws = [] noexcept { return 42; };

    const auto& as_const = readable;
    assert(as_const() == 42);
    assert(std::move(consumed)() == 42);
    assert(never_throws() == 42);
}
```



```
// The cv- and ref-qualifiers live in the *signature*, so the wrapper
// can promise what the target actually supports.
void qualifiers() {
    // Callable on a const object, because the target is const-callable.
    std::move_only_function<int() const> readable = [] { return 42; };

    // Callable once, on an rvalue.
    std::move_only_function<int() &&> consumed = [] { return 42; };

    // And noexcept is part of it too.
    std::move_only_function<int() const noexcept> never_throws = [] noexcept { return 42; };

    const auto& as_const = readable;
    assert(as_const() == 42);
    assert(std::move(consumed)() == 42);
    assert(never_throws() == 42);
}
```

```

// The cv- and ref-qualifiers live in the *signature*, so the wrapper
// can promise what the target actually supports.
void qualifiers() {
    // Callable on a const object, because the target is const-callable.
    std::move_only_function<int() const> readable = [] { return 42; };

    // Callable once, on an rvalue.
    std::move_only_function<int() &&> consumed = [] { return 42; };

    // And noexcept is part of it too.
    std::move_only_function<int() const noexcept> never_throws = [] noexcept { return 42; };

    const auto& as_const = readable;
    assert(as_const() == 42);
    assert(std::move(consumed)() == 42);
    assert(never_throws() == 42);
}

```

```
// The cv- and ref-qualifiers live in the *signature*, so the wrapper
// can promise what the target actually supports.
void qualifiers() {
    // Callable on a const object, because the target is const-callable.
    std::move_only_function<int() const> readable = [] { return 42; };

    // Callable once, on an rvalue.
    std::move_only_function<int() &&> consumed = [] { return 42; };

    // And noexcept is part of it too.
    std::move_only_function<int() const noexcept> never_throws = [] noexcept { return 42; };

    const auto& as_const = readable;
    assert(as_const() == 42);
    assert(std::move(consumed)() == 42);
    assert(never_throws() == 42);
}
```

What you give up

```
// What you give up: it cannot be copied, and there is no target() or  
// target_type() to ask what is inside.  
static_assert(!std::is_copy_constructible_v<std::move_only_function<int()>>);  
static_assert(std::is_move_constructible_v<std::move_only_function<int()>>);
```

What you give up

```
// What you give up: it cannot be copied, and there is no target() or  
// target_type() to ask what is inside.  
static_assert(!std::is_copy_constructible_v<std::move_only_function<int()>>);  
static_assert(std::is_move_constructible_v<std::move_only_function<int()>>);
```


What you give up

```
// What you give up: it cannot be copied, and there is no target() or
// target_type() to ask what is inside.
static_assert(!std::is_copy_constructible_v<std::move_only_function<int()>>);
static_assert(std::is_move_constructible_v<std::move_only_function<int()>>);
```

C++26: `std::copyable\_function`

```
// std::copyable_function (P2548) is std::function done again with  
// twenty years of hindsight: same copyability, but the qualifiers  
// work like move_only_function's, and the awkward corners of  
// std::function (target(), the allocator overloads, assignment from  
// a reference_wrapper) are gone.  
std::copyable_function<int() const> shareable = [] { return 42; };
```

C++26: `std::copyable\_function`

```
// std::copyable_function (P2548) is std::function done again with  
// twenty years of hindsight: same copyability, but the qualifiers  
// work like move_only_function's, and the awkward corners of  
// std::function (target(), the allocator overloads, assignment from  
// a reference_wrapper) are gone.
```

```
std::copyable_function<int() const> shareable = [] { return 42; };
```


C++26: `std::function\_ref`

```
// std::function_ref (P0792) is the one you want in a *parameter*: it
// refers to a callable it does not own, so there is no allocation and
// nothing to copy -- and nothing keeping the target alive either.
int apply(std::function_ref<int(int)> f, int n) {
    return f(n);
}
```

C++26: `std::function\_ref`

```
// std::function_ref (P0792) is the one you want in a *parameter*: it
// refers to a callable it does not own, so there is no allocation and
// nothing to copy -- and nothing keeping the target alive either.
int apply(std::function_ref<int(int)> f, int n) {
    return f(n);
}
```

C++26: `std::function\_ref`

```
// std::function_ref (P0792) is the one you want in a *parameter*: it
// refers to a callable it does not own, so there is no allocation and
// nothing to copy -- and nothing keeping the target alive either.
int apply(std::function_ref<int(int)> f, int n) {
    return f(n);
}
```

```

template <class> class function_ref;

template <class R, class... Args> class function_ref<R(Args...)> {
public:
    // Simplified: non-const lvalues only. The real std::function_ref
    // takes rather more care than this.
    template <class F>
    function_ref(F& f)
        : m_object{std::addressof(f)}
        , m_invoke{[](void* object, Args... args) → R {
            return (*static_cast<F*>(object))(args...);
        }} {}

    R operator()(Args... args) const { return m_invoke(m_object, args...); }

private:
    void* m_object;
    R (*m_invoke)(void*, Args...);
};

```

```

template <class> class function_ref;

template <class R, class... Args> class function_ref<R(Args...)> {
public:
    // Simplified: non-const lvalues only. The real std::function_ref
    // takes rather more care than this.
    template <class F>
    function_ref(F& f)
        : m_object{std::addressof(f)}
        , m_invoke{[](void* object, Args... args) → R {
            return (*static_cast<F*>(object))(args...);
        }} {}

    R operator()(Args... args) const { return m_invoke(m_object, args...); }

private:
    void* m_object;
    R (*m_invoke)(void*, Args...);
};

```



```

template <class> class function_ref;

template <class R, class... Args> class function_ref<R(Args...)> {
public:
    // Simplified: non-const lvalues only. The real std::function_ref
    // takes rather more care than this.
    template <class F>
    function_ref(F& f)
        : m_object{std::addressof(f)}
        , m_invoke{[](void* object, Args... args) → R {
            return (*static_cast<F*>(object))(args...);
        }} {}

    R operator()(Args... args) const { return m_invoke(m_object, args...); }

private:
    void* m_object;
    R (*m_invoke)(void*, Args...);
};

```

```

template <class> class function_ref;

template <class R, class... Args> class function_ref<R(Args...)> {
public:
    // Simplified: non-const lvalues only. The real std::function_ref
    // takes rather more care than this.
    template <class F>
    function_ref(F& f)
        : m_object{std::addressof(f)}
        , m_invoke{[](void* object, Args... args) → R {
            return (*static_cast<F*>(object))(args...);
        }} {}

    R operator()(Args... args) const { return m_invoke(m_object, args...); }

private:
    void* m_object;
    R (*m_invoke)(void*, Args...);
};

```

...and the hazard that comes with it

```
// The hazard is the same one a reference has always had. This is fine
// -- the temporary outlives the call...
int fine() {
    return apply([](int n) { return n * 2; }, 21);
}

// ...and this is not: the closure is gone before f is ever called.
// std::function_ref belongs in a parameter, not in a member.
std::function_ref<int(int)> dangling() {
    auto twice = [](int n) { return n * 2; };
    return twice; // returns a reference to a dead closure
}
```


...and the hazard that comes with it

```
// The hazard is the same one a reference has always had. This is fine
// -- the temporary outlives the call...
int fine() {
    return apply([](int n) { return n * 2; }, 21);
}

// ...and this is not: the closure is gone before f is ever called.
// std::function_ref belongs in a parameter, not in a member.
std::function_ref<int(int)> dangling() {
    auto twice = [](int n) { return n * 2; };
    return twice; // returns a reference to a dead closure
}
```

...and the hazard that comes with it

```
// The hazard is the same one a reference has always had. This is fine
// -- the temporary outlives the call...
int fine() {
    return apply([](int n) { return n * 2; }, 21);
}

// ...and this is not: the closure is gone before f is ever called.
// std::function_ref belongs in a parameter, not in a member.
std::function_ref<int(int)> dangling() {
    auto twice = [](int n) { return n * 2; };
    return twice; // returns a reference to a dead closure
}
```

Passing versus storing

The decision that actually matters

Erased

```
//    One instantiation, an indirect call, and possibly an allocation.  
int apply_erased(std::function<int(int)> f, int n) {  
    return f(n);  
}
```

Erased

```
//    One instantiation, an indirect call, and possibly an allocation.  
int apply_erased(std::function<int(int)> f, int n) {  
    return f(n);  
}
```


Templated

```
// One instantiation per callable; the call can inline.  
template <class F> int apply_template(F&& f, int n) {  
    return f(n);  
}
```

Templated

```
// One instantiation per callable; the call can inline.  
template <class F> int apply_template(F&& f, int n) {  
    return f(n);  
}
```

The version we shipped first

```
class BlockProcessor {
public:
    // Honest about the signature; hides an allocation and an indirect
    // call inside a real-time audio callback.
    void Process(std::span<const float> input,
                std::span<float> output,
                std::function<void(std::span<float>)> fn) {
        std::copy(input.begin(), input.end(), output.begin());
        fn(output);
    }
};
```


The version we shipped first

```
class BlockProcessor {
public:
    // Honest about the signature; hides an allocation and an indirect
    // call inside a real-time audio callback.
    void Process(std::span<const float> input,
                std::span<float> output,
                std::function<void(std::span<float>)> fn) {
        std::copy(input.begin(), input.end(), output.begin());
        fn(output);
    }
};
```

The version we shipped first

```
class BlockProcessor {
public:
    // Honest about the signature; hides an allocation and an indirect
    // call inside a real-time audio callback.
    void Process(std::span<const float> input,
                std::span<float> output,
                std::function<void(std::span<float>)> fn) {
        std::copy(input.begin(), input.end(), output.begin());
        fn(output);
    }
};
```

```
class TemplatedBlockProcessor {
public:
    // No allocation, no indirect call -- and no documentation either.
    template <class ProcessFn>
    void Process(std::span<const float> input, std::span<float> output, ProcessFn&& fn) {
        std::copy(input.begin(), input.end(), output.begin());
        fn(output);
    }
};
```

```
class TemplatedBlockProcessor {
public:
    // No allocation, no indirect call -- and no documentation either.
    template <class ProcessFn>
    void Process(std::span<const float> input, std::span<float> output, ProcessFn&& fn) {
        std::copy(input.begin(), input.end(), output.begin());
        fn(output);
    }
};
```

The version we have now

```
class ConstrainedBlockProcessor {  
public:  
    // The signature is back, and it is checked at the call site  
    // instead of somewhere inside the body.  
    void Process(std::span<const float> input,  
                std::span<float> output,  
                std::invocable<std::span<float>> auto&& fn) {  
        std::copy(input.begin(), input.end(), output.begin());  
        fn(output);  
    }  
}
```


The version we have now

```
class ConstrainedBlockProcessor {  
public:  
    // The signature is back, and it is checked at the call site  
    // instead of somewhere inside the body.  
    void Process(std::span<const float> input,  
                std::span<float> output,  
                std::invocable<std::span<float>> auto&& fn) {  
        std::copy(input.begin(), input.end(), output.begin());  
        fn(output);  
    }  
};
```

The concepts worth knowing

- `std::invocable<F, Args...>` — you can call it
- `std::regular_invocable<F, Args...>` — ...and it is *equality-preserving*, i.e. a real mathematical function
 - the difference is a **semantic** requirement; the compiler cannot check it
- `std::predicate<F, Args...>` — returns something boolean-testable
- `std::relation`, `std::equivalence_relation`, `std::strict_weak_order` — for comparators
- `std::invoke_result_t<F, Args...>` — what you get back

A decision tree

- **Passing it, and you can be a template?** — a constrained `auto&&` parameter.
`std::invocable<...> auto&&`
- **Passing it, and you cannot be a template?** — `function_ref` (C++26), or a template wrapper over a hand-rolled one today
- **Storing it, and it must be copyable?** — `copyable_function` (C++26), else `std::function`
- **Storing it, and it owns something?** — `move_only_function` (C++23)
- **Crossing a C ABI?** — `void(*) (void*)` plus a `void*`, the way every C API has always done it
- **Capture-less, and a C API wants a plain pointer?** — the lambda converts

Bugs I have shipped

All four of these were in production

```

// Stand-in for boost::function: a callable wrapper from before
// <functional> had one, with its own notion of "empty".
class legacy_function {
public:
    legacy_function() = default;

    legacy_function(void (*f)()) : m_f{f} {}

    explicit operator bool() const { return m_f != nullptr; }

    void operator()() const {
        if (m_f == nullptr) {
            throw std::runtime_error{"empty legacy_function"};
        }
        m_f();
    }

private:
    void (*m_f)(){};
};

```

```

// Stand-in for boost::function: a callable wrapper from before
// <functional> had one, with its own notion of "empty".
class legacy_function {
public:
    legacy_function() = default;

    legacy_function(void (*f)()) : m_f{f} {}

    explicit operator bool() const { return m_f != nullptr; }

    void operator()() const {
        if (m_f == nullptr) {
            throw std::runtime_error{"empty legacy_function"};
        }
        m_f();
    }

private:
    void (*m_f)(){};
};

```

```
// Stand-in for boost::function: a callable wrapper from before
// <functional> had one, with its own notion of "empty".
class legacy_function {
public:
    legacy_function() = default;

    legacy_function(void (*f)()) : m_f{f} {}

    explicit operator bool() const { return m_f != nullptr; }

    void operator()() const {
        if (m_f == nullptr) {
            throw std::runtime_error{"empty legacy_function"};
        }
        m_f();
    }

private:
    void (*m_f)(){};
};
```

```

// Stand-in for boost::function: a callable wrapper from before
// <functional> had one, with its own notion of "empty".
class legacy_function {
public:
    legacy_function() = default;

    legacy_function(void (*f)()) : m_f{f} {}

    explicit operator bool() const { return m_f != nullptr; }

    void operator()() const {
        if (m_f == nullptr) {
            throw std::runtime_error{"empty legacy_function"};
        }
        m_f();
    }

private:
    void (*m_f)(){};
};

```



```

// Stand-in for boost::function: a callable wrapper from before
// <functional> had one, with its own notion of "empty".
class legacy_function {
public:
    legacy_function() = default;

    legacy_function(void (*f)()) : m_f{f} {}

    explicit operator bool() const { return m_f != nullptr; }

    void operator()() const {
        if (m_f == nullptr) {
            throw std::runtime_error{"empty legacy_function"};
        }
        m_f();
    }

private:
    void (*m_f)(){};
};

```

```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```

```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```



```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```

```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```

```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```

```

// [func.wrap.func.con] gives a closed list of three things whose
// emptiness std::function propagates: a null function pointer, a null
// member pointer, and another std::function. Anything else -- however
// empty -- is just a target, and a target means "not empty".
bool the_trap() {
    auto legacy = legacy_function{}; // empty

    std::function<void()> callback = legacy;

    assert(!legacy); // it knows it is empty
    assert(callback); // ...and std::function does not
    assert(callback != nullptr);

    try {
        callback();
        return false;
    } catch (const std::bad_function_call&) {
        return false; // not this one either
    } catch (const std::runtime_error&) {
        return true; // the legacy wrapper's own exception escapes
    }
}

```

This is not a Boost problem

```
// This is not a Boost problem. It is what "propagate emptiness only
// for types I was told about" always does -- and the C++23 wrappers
// were told about each other, but not about std::function.
#if defined(__cpp_lib_move_only_function)
bool still_broken_in_23() {
    std::function<void()> empty;
    std::move_only_function<void()> wrapped{std::move(empty)};

    return static_cast<bool>(wrapped); // true, and calling it throws
}
#endif
```


This is not a Boost problem

```
// This is not a Boost problem. It is what "propagate emptiness only
// for types I was told about" always does -- and the C++23 wrappers
// were told about each other, but not about std::function.
#if defined(__cpp_lib_move_only_function)
bool still_broken_in_23() {
    std::function<void()> empty;
    std::move_only_function<void()> wrapped{std::move(empty)};

    return static_cast<bool>(wrapped); // true, and calling it throws
}
#endif
```

This is not a Boost problem

```
// This is not a Boost problem. It is what "propagate emptiness only
// for types I was told about" always does -- and the C++23 wrappers
// were told about each other, but not about std::function.
#if defined(__cpp_lib_move_only_function)
bool still_broken_in_23() {
    std::function<void()> empty;
    std::move_only_function<void()> wrapped{std::move(empty)};

    return static_cast<bool>(wrapped); // true, and calling it throws
}
#endif
```

This is not a Boost problem

```
// This is not a Boost problem. It is what "propagate emptiness only
// for types I was told about" always does -- and the C++23 wrappers
// were told about each other, but not about std::function.
#if defined(__cpp_lib_move_only_function)
bool still_broken_in_23() {
    std::function<void()> empty;
    std::move_only_function<void()> wrapped{std::move(empty)};

    return static_cast<bool>(wrapped); // true, and calling it throws
}
#endif
```


The fix is one line

```
// Ask the wrapper you have, not the one you are building.  
std::function<void()> safely(const legacy_function& legacy) {  
    if (!legacy) {  
        return nullptr;  
    }  
    return legacy;  
}
```

The fix is one line

```
// Ask the wrapper you have, not the one you are building.  
std::function<void()> safely(const legacy_function& legacy) {  
    if (!legacy) {  
        return nullptr;  
    }  
    return legacy;  
}
```

Bug two: by-reference capture, locally

```
// Used locally: capture by reference. It is cheaper, the side effect
// on `total` is the point, and the lambda cannot outlive anything.
int sum_of_squares(const std::vector<int>& v) {
    int total = 0;

    std::for_each(v.begin(), v.end(), [&total](int n) { total += n * n; });

    return total;
}
```

Bug two: by-reference capture, locally

```
// Used locally: capture by reference. It is cheaper, the side effect
// on `total` is the point, and the lambda cannot outlive anything.
int sum_of_squares(const std::vector<int>& v) {
    int total = 0;

    std::for_each(v.begin(), v.end(), [&total](int n) { total += n * n; });

    return total;
}
```

...and by-reference capture, escaping

```
// Stored, returned or handed to another thread: by-reference capture
// is a reference member on an object that outlives its referent.
//
//     std::function<int()> counter_for_the_UI() {
//         int count = 0;
//         return [&count] { return ++count; };    // dangles on return
//     }
//
// The closure is fine. The reference in it is not. Nothing in the
// type system stops you, and nothing at the call site looks wrong.
```


...and by-reference capture, escaping

```
// Stored, returned or handed to another thread: by-reference capture
// is a reference member on an object that outlives its referent.
//
//     std::function<int()> counter_for_the_UI() {
//         int count = 0;
//         return [&count] { return ++count; };    // dangles on return
//     }
//
// The closure is fine. The reference in it is not. Nothing in the
// type system stops you, and nothing at the call site looks wrong.
```

...and by-reference capture, escaping

```
// Stored, returned or handed to another thread: by-reference capture
// is a reference member on an object that outlives its referent.
//
//     std::function<int()> counter_for_the_UI() {
//         int count = 0;
//         return [&count] { return ++count; };    // dangles on return
//     }
//
// The closure is fine. The reference in it is not. Nothing in the
// type system stops you, and nothing at the call site looks wrong.
```

If it escapes, it owns

```
// If it escapes, it owns what it needs.  
std::function<int()> counter_for_the_ui() {  
    return [count = 0]() mutable { return ++count; };  
}
```


If it escapes, it owns

```
// If it escapes, it owns what it needs.  
std::function<int()> counter_for_the_ui() {  
    return [count = 0]() mutable { return ++count; };  
}
```

The one that actually got us

```
// The one that actually got us: the capture is by value, and the
// *pointee* is what dangled.
std::function<int()> size_reporter(const std::vector<int>& v) {
    // `v` is a reference parameter; `&v` is captured by value and is
    // no better for it.
    return [p = &v] { return static_cast<int>(p->size()); };
}
```

The one that actually got us

```
// The one that actually got us: the capture is by value, and the
// *pointee* is what dangled.
std::function<int()> size_reporter(const std::vector<int>& v) {
    // `v` is a reference parameter; `&v` is captured by value and is
    // no better for it.
    return [p = &v] { return static_cast<int>(p->size()); };
}
```

The one that actually got us

```
// The one that actually got us: the capture is by value, and the
// *pointee* is what dangled.
std::function<int()> size_reporter(const std::vector<int>& v) {
    // `v` is a reference parameter; `&v` is captured by value and is
    // no better for it.
    return [p = &v] { return static_cast<int>(p->size()); };
}
```

Bug three: the observer owns the observable

```
// The shape every observable has: it owns its observer list, and the
// observers are type-erased callables.
struct Model {
    std::vector<std::function<void(float)>> onValueChanged;
    float last{};

    ~Model() { ++models_destroyed; }

    void Set(float value) {
        for (auto& observer : onValueChanged) {
            observer(value);
        }
    }
};
```


Bug three: the observer owns the observable

```
// The shape every observable has: it owns its observer list, and the
// observers are type-erased callables.
struct Model {
    std::vector<std::function<void(float)>> onValueChanged;
    float last{};

    ~Model() { ++models_destroyed; }

    void Set(float value) {
        for (auto& observer : onValueChanged) {
            observer(value);
        }
    }
};
```

Bug three: the observer owns the observable

```
// The shape every observable has: it owns its observer list, and the
// observers are type-erased callables.
struct Model {
    std::vector<std::function<void(float)>> onValueChanged;
    float last{};

    ~Model() { ++models_destroyed; }

    void Set(float value) {
        for (auto& observer : onValueChanged) {
            observer(value);
        }
    }
};
```

Follow the arrows

```
// The bug. Follow the arrows:
//
//   Model → onValueChanged → std::function → shared_ptr → Model
//
// The strong count never reaches zero, so the destructor never runs
// and nothing in the code looks wrong.
std::weak_ptr<Model> leaky() {
    auto model = std::make_shared<Model>();

    model→onValueChanged.push_back([model](float value) { model→last = value; });

    return model;
}
```


Follow the arrows

```
// The bug. Follow the arrows:
//
//   Model → onValueChanged → std::function → shared_ptr → Model
//
// The strong count never reaches zero, so the destructor never runs
// and nothing in the code looks wrong.
std::weak_ptr<Model> leaky() {
    auto model = std::make_shared<Model>();

    model→onValueChanged.push_back([model](float value) { model→last = value; });

    return model;
}
```

Follow the arrows

```
// The bug. Follow the arrows:  
//  
//   Model → onValueChanged → std::function → shared_ptr → Model  
//  
// The strong count never reaches zero, so the destructor never runs  
// and nothing in the code looks wrong.  
std::weak_ptr<Model> leaky() {  
    auto model = std::make_shared<Model>();  
  
    model→onValueChanged.push_back([model](float value) { model→last = value; });  
  
    return model;  
}
```

Follow the arrows

```
// The bug. Follow the arrows:
//
//   Model → onValueChanged → std::function → shared_ptr → Model
//
// The strong count never reaches zero, so the destructor never runs
// and nothing in the code looks wrong.
std::weak_ptr<Model> leaky() {
    auto model = std::make_shared<Model>();

    model→onValueChanged.push_back([model](float value) { model→last = value; });

    return model;
}
```

Fix one: observe, do not own

```
// Fix one: observe, do not own. The callback needs the model to be
// alive, but it must not be what keeps it alive.
std::weak_ptr<Model> weakly() {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([weak = std::weak_ptr{model}](float value) {
        if (auto locked = weak.lock()) {
            locked->last = value;
        }
    });

    return model;
}
```

Fix one: observe, do not own

```
// Fix one: observe, do not own. The callback needs the model to be
// alive, but it must not be what keeps it alive.
std::weak_ptr<Model> weakly() {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([weak = std::weak_ptr{model}](float value) {
        if (auto locked = weak.lock()) {
            locked->last = value;
        }
    });

    return model;
}
```


Fix one: observe, do not own

```
// Fix one: observe, do not own. The callback needs the model to be
// alive, but it must not be what keeps it alive.
std::weak_ptr<Model> weakly() {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([weak = std::weak_ptr{model}](float value) {
        if (auto locked = weak.lock()) {
            locked->last = value;
        }
    });

    return model;
}
```

```
// Fix two, which is what we actually shipped: capture only what the
// callback uses, and give the model a single owner somewhere else --
// the widget it belongs to. When the widget dies, so does the model.
struct Widget {
    std::shared_ptr<Model> owned;
    float shown{};
};

std::weak_ptr<Model> rehomed(Widget& widget) {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([&widget](float value) { widget.shown = value; });

    widget.owned = std::move(model);
    return widget.owned;
}
```

```

// Fix two, which is what we actually shipped: capture only what the
// callback uses, and give the model a single owner somewhere else --
// the widget it belongs to. When the widget dies, so does the model.
struct Widget {
    std::shared_ptr<Model> owned;
    float shown{};
};

std::weak_ptr<Model> rehomed(Widget& widget) {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([&widget](float value) { widget.shown = value; });

    widget.owned = std::move(model);
    return widget.owned;
}

```



```
// Fix two, which is what we actually shipped: capture only what the
// callback uses, and give the model a single owner somewhere else --
// the widget it belongs to. When the widget dies, so does the model.
struct Widget {
    std::shared_ptr<Model> owned;
    float shown{};
};

std::weak_ptr<Model> rehomed(Widget& widget) {
    auto model = std::make_shared<Model>();

    model->onValueChanged.push_back([&widget](float value) { widget.shown = value; });

    widget.owned = std::move(model);
    return widget.owned;
}
```

Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);       // and so is the original's  
  
    return stored();  
}
```

Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);        // and so is the original's  
  
    return stored();  
}
```

Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);        // and so is the original's  
  
    return stored();  
}
```

Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);       // and so is the original's  
  
    return stored();  
}
```


Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);       // and so is the original's  
  
    return stored();  
}
```

Bug four: a copy has its own state

```
int copies() {  
    auto next = [n = 0]() mutable { return ++n; };  
  
    std::function<int()> stored = next; // copy  
    auto also_stored = stored;         // copy of the copy  
  
    assert(stored() == 1);  
    assert(stored() == 2);  
    assert(also_stored() == 1); // its own n  
    assert(next() == 1);       // and so is the original's  
  
    return stored();  
}
```

...and every algorithm copies it

```
struct Counting {  
    int calls{};  
  
    void operator()(int) { ++calls; }  
};  
  
int count_via_for_each(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // for_each takes the callable by value. The only reason you can  
    // see what it counted is that it hands the copy back.  
    const auto used = std::for_each(v.begin(), v.end(), counter);  
  
    assert(counter.calls == 0); // ...ours never ran
```


...and every algorithm copies it

```
struct Counting {  
    int calls{};  
  
    void operator()(int) { ++calls; }  
};  
  
int count_via_for_each(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // for_each takes the callable by value. The only reason you can  
    // see what it counted is that it hands the copy back.  
    const auto used = std::for_each(v.begin(), v.end(), counter);  
  
    assert(counter.calls == 0); // ...ours never ran
```

...and every algorithm copies it

```
struct Counting {  
    int calls{};  
  
    void operator()(int) { ++calls; }  
};  
  
int count_via_for_each(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // for_each takes the callable by value. The only reason you can  
    // see what it counted is that it hands the copy back.  
    const auto used = std::for_each(v.begin(), v.end(), counter);  
  
    assert(counter.calls == 0); // ...ours never ran
```

...and every algorithm copies it

```
struct Counting {  
    int calls{};  
  
    void operator()(int) { ++calls; }  
};  
  
int count_via_for_each(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // for_each takes the callable by value. The only reason you can  
    // see what it counted is that it hands the copy back.  
    const auto used = std::for_each(v.begin(), v.end(), counter);  
  
    assert(counter.calls == 0); // ...ours never ran
```

`std::ref` is the escape hatch

```
int count_via_ref(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // std::ref is the escape hatch: a copyable handle to the object  
    // you actually want called. (It is also the one thing  
    // std::function is guaranteed not to allocate for.)  
    std::for_each(v.begin(), v.end(), std::ref(counter));  
  
    return counter.calls;  
}
```

`std::ref` is the escape hatch

```
int count_via_ref(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // std::ref is the escape hatch: a copyable handle to the object  
    // you actually want called. (It is also the one thing  
    // std::function is guaranteed not to allocate for.)  
    std::for_each(v.begin(), v.end(), std::ref(counter));  
  
    return counter.calls;  
}
```


`std::ref` is the escape hatch

```
int count_via_ref(const std::vector<int>& v) {  
    auto counter = Counting{};  
  
    // std::ref is the escape hatch: a copyable handle to the object  
    // you actually want called. (It is also the one thing  
    // std::function is guaranteed not to allocate for.)  
    std::for_each(v.begin(), v.end(), std::ref(counter));  
  
    return counter.calls;  
}
```

```
// MUST FAIL: a lambda converts to a function pointer *and* to a  
// std::function, both by a user-defined conversion, so an overload  
// set offering both has no best match.
```

```
#include <functional>
```

```
void listen(void (*callback)());  
void listen(std::function<void()> callback);
```

```
int main() {  
    listen([] {}); // error: call to 'listen' is ambiguous  
}
```

```
// MUST FAIL: a lambda converts to a function pointer *and* to a  
// std::function, both by a user-defined conversion, so an overload  
// set offering both has no best match.
```

```
#include <functional>
```

```
void listen(void (*callback)());  
void listen(std::function<void()> callback);
```

```
int main() {  
    listen([] {}); // error: call to 'listen' is ambiguous  
}
```



```
// MUST FAIL: a lambda converts to a function pointer *and* to a  
// std::function, both by a user-defined conversion, so an overload  
// set offering both has no best match.
```

```
#include <functional>
```

```
void listen(void (*callback)());  
void listen(std::function<void()> callback);
```

```
int main() {  
    listen([] {}); // error: call to 'listen' is ambiguous  
}
```

Recommendations

What I would tell my team

- **Prefer a function.** F.50: use a lambda when a function *won't* do
- **Name it if it is reused.** F.10, F.11
- **Capture explicitly.** Never [=]; [&] only for a lambda used right here (F.52, F.53, F.54)
- **Template to pass, erase to store.** And when you erase, pick the wrapper whose signature says what you mean
- **Constrain the template.** `std::invocable<...> auto&&` costs nothing and documents everything
- **Never if (wrapper) on a wrapper you did not build.** Check the one you have

What the Core Guidelines does **not** say

- **`std::function` is never the subject of a rule**
 - it appears twice in the whole document, both in passing
- T.5's exceptions: *"Type erasure is sometimes appropriate, such as for `std::function`."*
- And in the unclassified proto-rules:
 - *"Should there be guidelines to choose between polymorphisms? YES. classic (virtual functions, reference semantics) vs. Sean Parent style (value semantics, type-erased, kind of like `std::function`) vs. CRTP/static? YES"*
- So: no guidance on passing, storing or returning one, and no rule about what type erasure costs

What is changing

- **C++23** — `move_only_function`, `static operator()`, deducing-this lambdas, `bind_back`, `invoke_r`
- **C++26** — `copyable_function`, `function_ref`
 - and a naming question still settling: the `nontype` tag from P0792 looks like it became `std::constant_wrapper` / `std::cw` — check the published IS, not cppreference
- **Still missing** — a standard answer for "callable, non-owning, const-correct, in a member"
- **Not coming** — a fix for `std::function`. It is what it is now

std::invokable

Bonus: Objective-C blocks

...because half of you ship on a Mac

A block literal

```
// ^ where a lambda would have [], and the type uses ^ where a
// function pointer uses *.
void (^hello)(void) = ^{
    printf("hello, CppCon\n");
};

hello();

typedef int (^Transform)(int);    // int (^)(int)
```


Captures: by value, and const

```
int x = 40;
```

```
int (^add_x)(int) = ^(int n) { return n + x; };
```

```
x = 0;
```

```
add_x(2);           // 42 -- the block has its own copy
```

```
// ...and it is const inside, exactly like [x] in C++:
```

```
void (^bump)(void) = ^{ ++x; }; // error: variable is not assignable
```

`\_\_block`: the mutable shared capture

```
__block int count = 0;
```

```
void (^bump)(void) = ^{ ++count; };
```

```
bump();
```

```
bump();
```

```
// count = 2 -- the *variable* moved into shared storage
```

Lifetime: a block literal is on the stack

```
typedef void (^Callback)(void);

Callback make_callback(void) {
    int n = 42;

    return ^{ printf("%d\n", n); };
    // MRR:  undefined -- the block lives in this frame
    // ARC:  fine      -- returning it copies it to the heap
}

// Pre-ARC, and in plain C++ where ARC does not apply:
#include <Block.h>

Callback keep(int n) {
    return Block_copy(^{ printf("%d\n", n); });
}

// ...and every Block_copy needs a Block_release.
```

Capturing `self` is the cycle from twenty slides ago

```
// Cycle: self owns the block, the block retains self.
self.onChange = ^{ [self update]; };

// Break it the same way we broke the shared_ptr cycle:
__weak typeof(self) weakSelf = self;
self.onChange = ^{
    __strong typeof(self) strongSelf = weakSelf;    // lock()
    if (strongSelf) { [strongSelf update]; }
};
```

Blocks and C++, side by side

- `^{} is [=]` — every mentioned local, by value, const
 - no capture list; `__block` is the only escape
- A block literal lives on the **stack**; storing it needs `Block_copy` (or ARC)
- A C++ closure is an **ordinary object** with ordinary lifetime
- A block **is** callable from C++ and converts to `std::function`
 - libc++ special-cases blocks in `std::function`'s emptiness check — the courtesy it never extended to `boost::function`
- Capturing `self` retains; capturing `shared_ptr` retains. Same bug, same fix

Thank you!

Ask me about AI!

Roth Michaels

Director of Software Research

roth.michaels@native-instruments.com

@thevibesman

